

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA KỸ THUẬT ĐIỆN TỬ 1



BÀI TẬP LỚN THIẾT KẾ HỆ THỐNG VLSI

Họ và tên	Mã sinh viên
Đỗ Nhật Thành	B22DCDT300
Bùi Hồng Quân	B22DCDT244

Hà Nội – 2026

MỤC LỤC

CHƯƠNG 1: GIỚI THIỆU VỀ BÀI TẬP LỚN.....	4
1.1. Mục tiêu bài tập lớn.	4
1.2. Vai trò của bộ cộng trong hệ thống số.	4
1.3. Các kiến trúc bộ cộng được khảo sát.	5
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT.	7
2.1 Khái niệm Generate và Propagate.....	7
2.2 Ripple-Carry Adder (RCA).....	7
2.3 Carry-Skip Adder (CskA) – Bộ cộng nhảy số dư.	8
2.4 Carry – Select Adder (CselA) – Bộ cộng chọn số dư.	9
2.5 Carry-Lookahead Adder (CLA) – Bộ cộng nhìn trước số dư.....	10
2.6 Parallel Prefix Adder (PPA) – Bộ cộng tiền tố song song.....	11
CHƯƠNG 3: THIẾT KẾ VERILOG.....	13
3.1 Ripple-Carry Adder.....	13
3.1.1 Mô tả kiến trúc.	13
3.1.2 Sơ đồ khối.	13
3.1.3 Các tham số thiết kế.	14
3.1.4 Cách tổ chức module.....	14
3.2 Carry-Skip Adder.	14
3.2.1 Mô tả kiến trúc.	14
3.2.2 Sơ đồ khối.	15
3.2.3 Các tham số thiết kế.	15
3.2.4 Cách tổ chức module.....	15
3.3 Carry-Select Adder.....	16
3.3.1 Mô tả kiến trúc.	16
3.3.2 Sơ đồ khối.	16
3.3.3 Các tham số thiết kế.	16
3.3.4 Cách tổ chức module.....	17
3.4 Carry-Lookahead Adder.....	17
3.4.1 Mô tả kiến trúc.	17

3.4.2 Sơ đồ khối.	17
3.4.3 Các tham số thiết kế.	18
3.4.4 Cách tổ chức module.....	18
3.5 Parallel Prefix Adder (Kogge–Stone).	18
3.5.1 Mô tả kiến trúc.	18
3.5.2 Sơ đồ khối.	19
3.5.3 Các tham số thiết kế.	19
3.5.4 Cách tổ chức module.....	20
CHƯƠNG 4: MÔ PHỎNG VÀ KIỂM THỬ.....	21
4.1 Mô tả testbench.	21
4.2 Các trường hợp kiểm thử và kết quả mô phỏng.....	21
4.3 Kết luận sau khi thu được kết quả mô phỏng.....	37
CHƯƠNG 5: TỔNG HỢP VÀ ĐÁNH GIÁ TRÊN FPGA.....	38
5.1 Nền tảng FPGA và Công cụ sử dụng.	38
5.2 Kết quả tổng hợp.	38
CHƯƠNG 6: SO SÁNH VÀ THẢO LUẬN	41
6.1 Về độ trễ (Delay).....	41
6.2 Về tài nguyên phần cứng.....	41
6.3 Về tần số hoạt động cực đại (Fmax).	41
6.4 Về công suất (Power).	42
6.5 Về tối ưu hóa đặc thù trên FPGA (FPGA-specific optimization).....	42

CHƯƠNG 1: GIỚI THIỆU VỀ BÀI TẬP LỚN

1.1. Mục tiêu bài tập lớn.

Trong các hệ thống số hiện đại, phép cộng là một trong những phép toán cơ bản và xuất hiện trong hầu hết các khối xử lý dữ liệu như ALU, DSP, CPU, GPU và các hệ thống nhúng. Hiệu năng của bộ cộng ảnh hưởng trực tiếp đến tốc độ hoạt động của toàn hệ thống.

Mục tiêu của bài tập lớn là:

- Tìm hiểu nguyên lý hoạt động của các kiến trúc bộ cộng số.
- Thiết kế các bộ cộng bằng ngôn ngữ mô tả phần cứng Verilog.
- Mô phỏng và kiểm chứng hoạt động của các bộ cộng.
- Tổng hợp thiết kế trên FPGA.
- Đánh giá và so sánh các kiến trúc dựa trên: độ trễ (delay), tần số hoạt động cực đại (fmax), tài nguyên phần cứng (LUT, FF), công suất tiêu thụ (power).

1.2. Vai trò của bộ cộng trong hệ thống số.

Bộ cộng là thành phần cơ bản trong các hệ thống xử lý số. Bộ cộng được sử dụng trong:

- Bộ xử lý trung tâm CPU.
- Bộ xử lý tín hiệu số DSP.
- Bộ nhân số học.
- Bộ cộng dồn MAC.
- Bộ tính địa chỉ bộ nhớ.
- Khối xử lý hình ảnh và AI accelerator.

Hiệu năng của bộ cộng ảnh hưởng trực tiếp đến:

- Tốc độ xử lý dữ liệu.
- Độ trễ hệ thống.
- Tần số hoạt động tối đa.
- Mức tiêu thụ công suất.
- Diện tích phần cứng.

→ Do đó việc nghiên cứu các kiến trúc bộ cộng là nội dung rất quan trọng trong thiết kế vi mạch số và FPGA.

1.3. Các kiến trúc bộ cộng được khảo sát.

Ripple-Carry Adder

Đây là kiến trúc đơn giản nhất. Carry được truyền tuần tự từ bit thấp đến bit cao.

Ưu điểm:

- Thiết kế đơn giản.
- Ít tài nguyên phần cứng.

Nhược điểm:

- Độ trễ lớn khi số bit tăng.

Carrrt-Skip Adder

Kiến trúc này chia bộ cộng thành nhiều block và cho phép carry bỏ qua một block nếu tất cả bit propagate.

Ưu điểm:

- Nhanh hơn RCA.
- Tài nguyên vừa phải.

Nhược điểm:

- Logic điều khiển phức tạp hơn.

Carry-Select Adder

Kiến trúc này tính toán song song với hai trường hợp carry vào bằng 0 và bằng 1.

Ưu điểm:

- Giảm đáng kể độ trễ.

Nhược điểm:

- Tốn nhiều tài nguyên phần cứng.

Carry-Lookahead Adder.

CLA sử dụng tín hiệu generate và propagate để tính carry song song.

Ưu điểm:

- Tốc độ cao.

Nhược điểm:

- Logic phức tạp hơn RCA.

Paralleled Prefix Adder

Đây là nhóm bộ cộng tốc độ cao sử dụng cây prefix để tính carry.

Ưu điểm:

- Độ trễ rất nhỏ.
- Tốc độ cao.

Nhược điểm:

- Tốn nhiều LUT và routing.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT.

2.1 Khái niệm Generate và Propagate.

Đây là nền tảng toán học để tối ưu hóa việc tính toán số dư (carry) thay vì đợi nó lan truyền tuần tự. Xét hai bit đầu vào A_i và B_i :

- **Generate (G_i):** tín hiệu tạo dư, số dư được tạo ra tại vị trí i bất kể số dư đầu vào là gì.

$$G_i = A_i \cdot B_i$$

- **Propagate (P_i):** tín hiệu lan truyền, số dư từ vị trí trước (C_{i-1}) sẽ được truyền qua vị trí i .

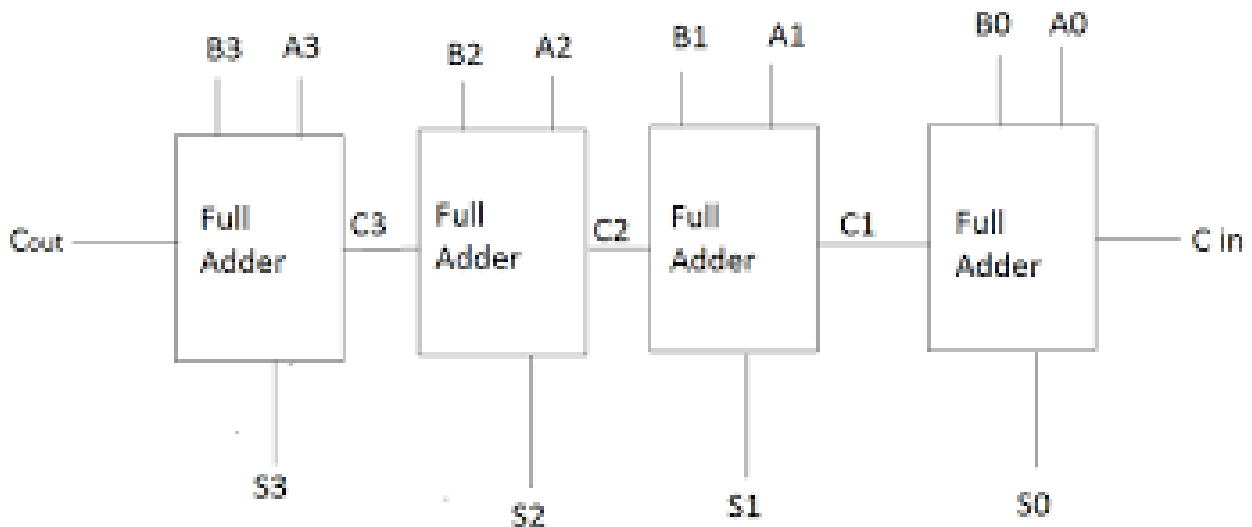
$$P_i = A_i \oplus B_i \text{ (hoặc } A_i + B_i)$$

- **Tổng (S_i) và số dư ($C_{out,i}$):**

$$S_i = P_i \oplus C_{i-1}$$

$$C_i = G_i + P_i \cdot C_{i-1}$$

2.2 Ripple-Carry Adder (RCA).



RCA được tạo thành bằng cách nối các bộ Full Adder (FA) lại với nhau.

Nguyên lý: Cout của FA thứ i là Cin của FA thứ $i + 1$.

Độ trễ (Critical Path): Nếu có bộ cộng n-bit, độ trễ sẽ là $t_{RCA} = n \times t_{FA}$ (trong đó t_{FA} là thời gian trễ của một bộ FA).

Đánh giá: Rất tốn thời gian khi n lớn (ví dụ 32 hay 64 bit) vì bit thứ 6 phải đợi bit thứ 1 tính xong.

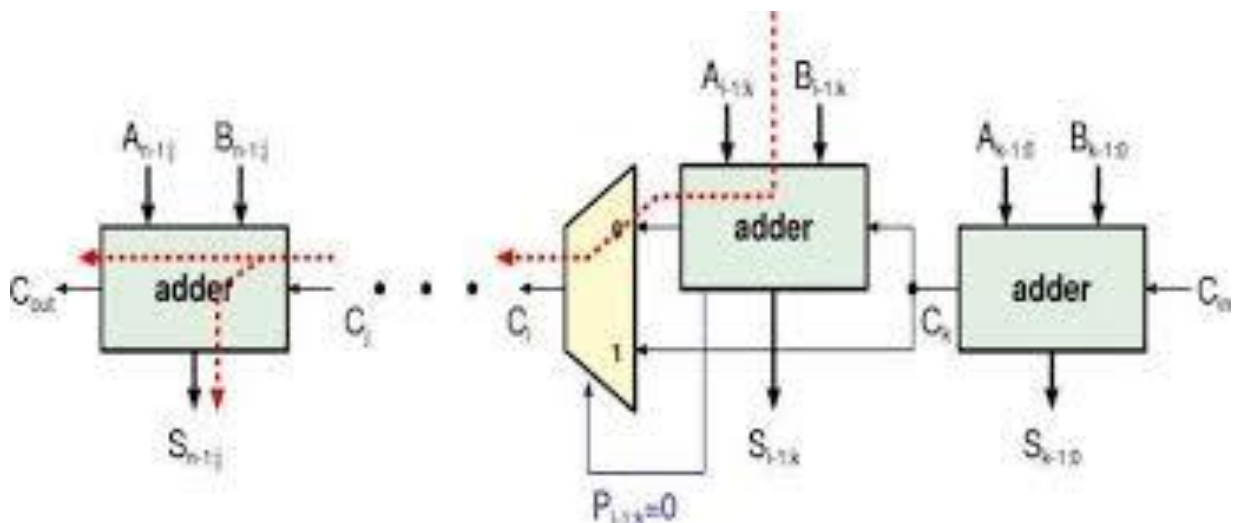
Ưu điểm:

- Cấu trúc cực kỳ đơn giản, dễ thiết kế và kiểm tra.
- Diện tích phần cứng nhỏ nhất ($O(n)$) do chỉ sử dụng các khối Full Adder (FA) nối tiếp.
- Tiêu thụ công suất thấp ở tần số thấp.

Nhược điểm:

- Tốc độ rất chậm: Số dư phải “chạy” qua từng bit. Độ trễ tăng tuyến tính theo số bit ($O(n)$).
- Không phù hợp cho các hệ thống xử lý tốc độ cao như CPU hay DSP hiện đại.

2.3 Carry-Skip Adder (CskA) – Bộ cộng nhảy số dư.



Cấu trúc: Chia n bit thành các khối k bit (ví dụ khối 4-bit).

Cơ chế: Nếu tất cả các bit trong khối đều có tín hiệu $P = 1$, tức là $P_{block} = P_0.P_1.P_2.P_3 = 1$, thì số dư đầu vào khối sẽ được đưa thẳng ra đầu ra khối qua một cổng MUX.

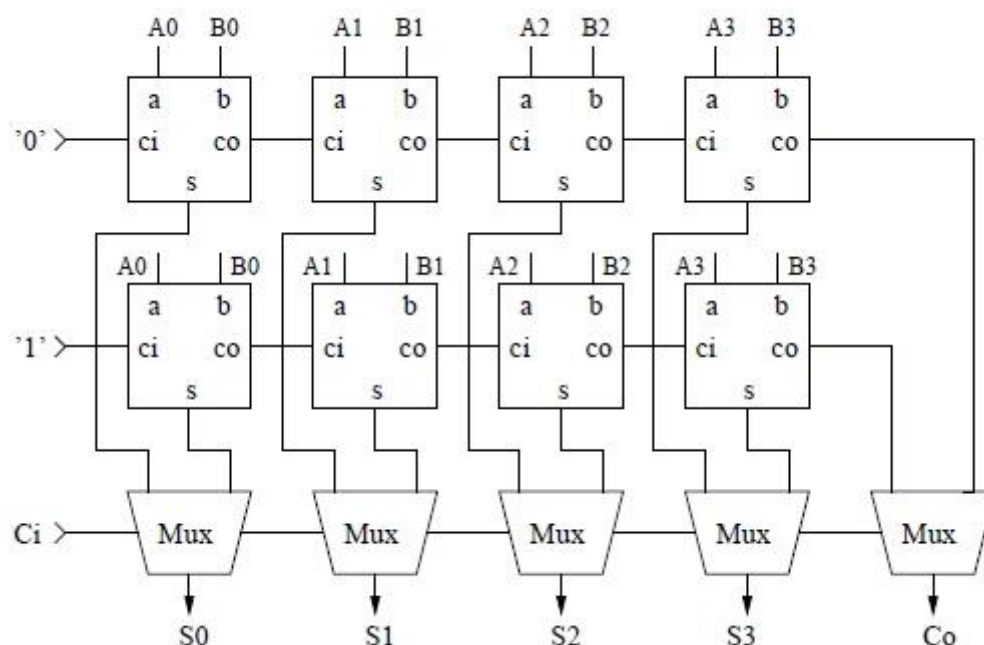
Ưu điểm:

- Cải thiện tốc độ đáng kể so với RCA với chi phí diện tích tăng thêm rất ít (chỉ thêm một vài cổng AND và MUX).
- Hiệu quả cao đối với các phép cộng mà số dư thường xuyên được lan truyền (propagate).

Nhược điểm:

- Độ trễ vẫn phụ thuộc vào cấu trúc RCA bên trong các khối.
- Việc chia kích thước khối (block size) không hợp lý sẽ làm giảm hiệu năng đáng kể.
- Vẫn chậm hơn nhiều so với CLA hay PPA.

2.4 Carry – Select Adder (CselA) – Bộ cộng chọn số dư.



Cấu trúc: Mỗi khối sẽ gồm hai bộ RCA chạy song song:

- Nhánh 1: Tính tổng với giả định $C_{in} = 0$.

- Nhánh 2: Tính tổng với giả định $C_{in} = 1$.

Cơ chế: Khi số dư thực sự từ khối trước truyền tới, nó sẽ đóng vai trò là tín hiệu điều khiển MUX để chọn kết quả đúng trong hai nhánh đã tính sẵn.

Độ trễ: Rất nhanh, chỉ phụ thuộc vào thời gian trễ của MUX ở các tầng cuối.

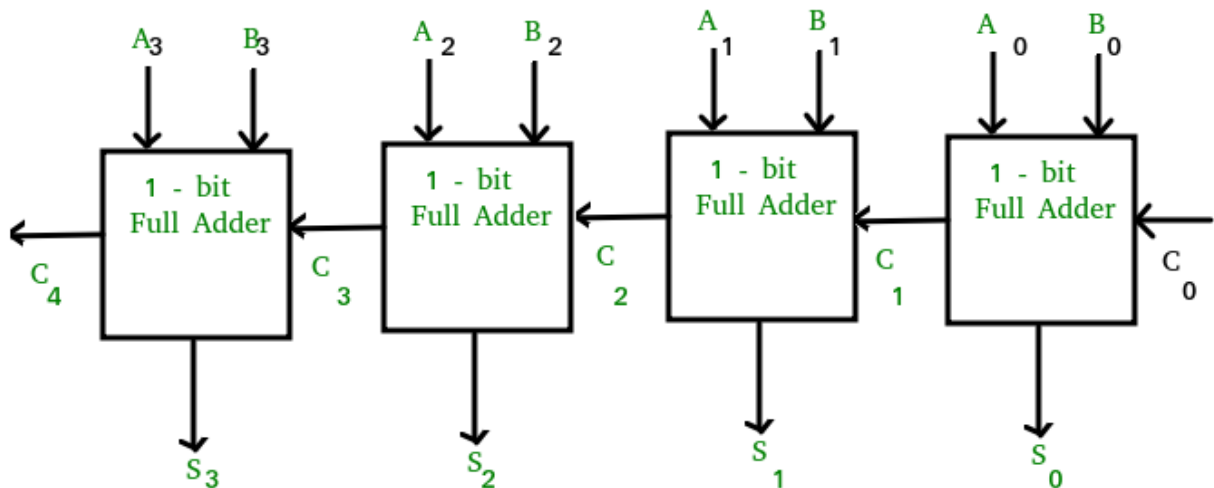
Ưu điểm:

- Tốc độ nhanh hơn nhiều so với RCA và CskA vì các kết quả tổng tiềm năng được tính toán song song.
- Độ trễ chỉ còn phụ thuộc vào thời gian chuyển mạch của bộ MUX.

Nhược điểm:

- Tốn diện tích: Đây là nhược điểm lớn nhất vì phải nhân đôi số lượng bộ cộng (một bộ cho trường hợp $C_{in} = 0$, một cho $C_{in} = 1$).
- Công suất tiêu thụ cao do có nhiều cổng logic hoạt động cùng lúc.

2.5 Carry-Lookahead Adder (CLA) – Bộ cộng nhìn trước số dư.



Đây là bộ cộng giải quyết triệt để bài toán đợi chờ bằng cách triển khai các phương trình đại số Boole.

Phương trình tổng quát:

- $C_1 = G_0 + P_0C_{in}$
- $C_2 = G_1 + P_1G_0 + P_1P_0C_{in}$
- $C_3 = G_2 + P_2G_1 + P_2P_1G_0 + P_2P_1P_0C_{in}$

Cấu trúc: Sử dụng khối Lookahead Carry Unit (LCU) để tính toán tất cả C_i cùng một lúc dựa trên A và B.

Giới hạn: Khi số bit tăng lên, số lượng cổng AND/OR đầu vào lớn (fan-in) tăng cực nhanh, làm mạch trở nên cồng kềnh và khó chế tạo trên chip.

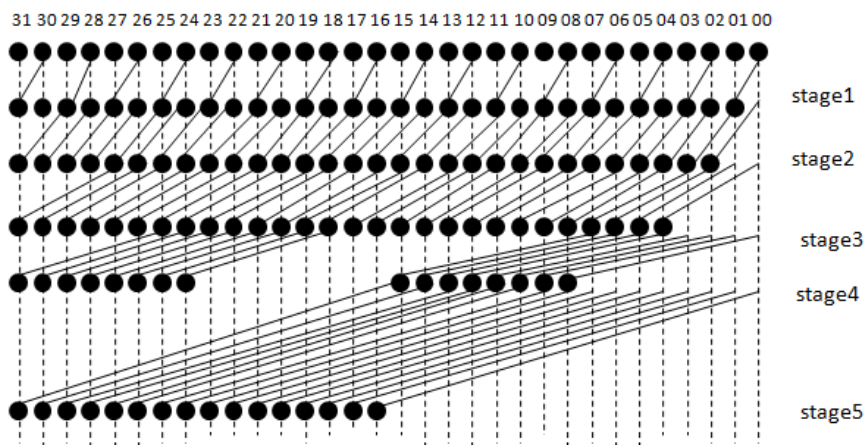
Ưu điểm:

- Tốc độ cực nhanh cho các bộ cộng kích thước nhỏ (4-bit, 8-bit).
- Độ trễ không còn phụ thuộc vào sự lan truyền từng bit mà tính toán trực tiếp từ đầu vào.

Nhược điểm:

- Độ phức tạp phần cứng: Khi số bit tăng lên, số lượng ngõ vào của các cổng logic tăng quá lớn, dẫn đến việc khó hiện thực hóa trên chip.

2.6 Parallel Prefix Adder (PPA) – Bộ cộng tiền tố song song.



Đây là sự mở rộng của CLA sang cấu trúc cây phân cấp, giúp tối ưu hóa diện tích và tốc độ cho các hệ thống 32-bit hoặc 64-bit.

Nguyên lý: Sử dụng toán tử tiền tố để kết hợp các cặp (G, P):

$$(g, p) \circ (g', p') = (g + (p \cdot g'), p \cdot p')$$

Các kiến trúc tiêu biểu:

- **Kogge-Stone Adder (KSA):** Có độ trễ thấp nhất $\log_2 n$, dây nối cực kỳ dày đặc (Wiring congestion), thường dùng cho CPU hiệu năng cao.
- **Brent-Kung Adder (BKA):** Tiết kiệm diện tích hơn, dây nối thưa hơn nhưng độ trễ lớn hơn $(2\log_2 n - 1)$.
- **Ladner-Fischer:** Cân bằng giữa diện tích và tốc độ.

Ưu điểm:

- Tốc độ tối ưu nhất: Đạt độ trễ mức lô-ga-rit ($O(\log n)$), cực kỳ lý tưởng cho các bộ vi xử lý 32-bit, 64-bit hoặc 128-bit.
- Cấu trúc phân cấp giúp dễ dàng module hóa thiết kế.

Nhược điểm:

- Diện tích rất lớn: Cần rất nhiều node tính toán trung gian (các cổng prefix).
- Wiring Congestion (Tắc nghẽn đường dây): Do các tín hiệu phải kết nối chéo giữa các tầng, việc đi dây trên chip trở nên cực kỳ phức tạp, có thể gây nhiễu và tăng công suất tiêu thụ do điện dung đường dây.

CHƯƠNG 3: THIẾT KẾ VERILOG

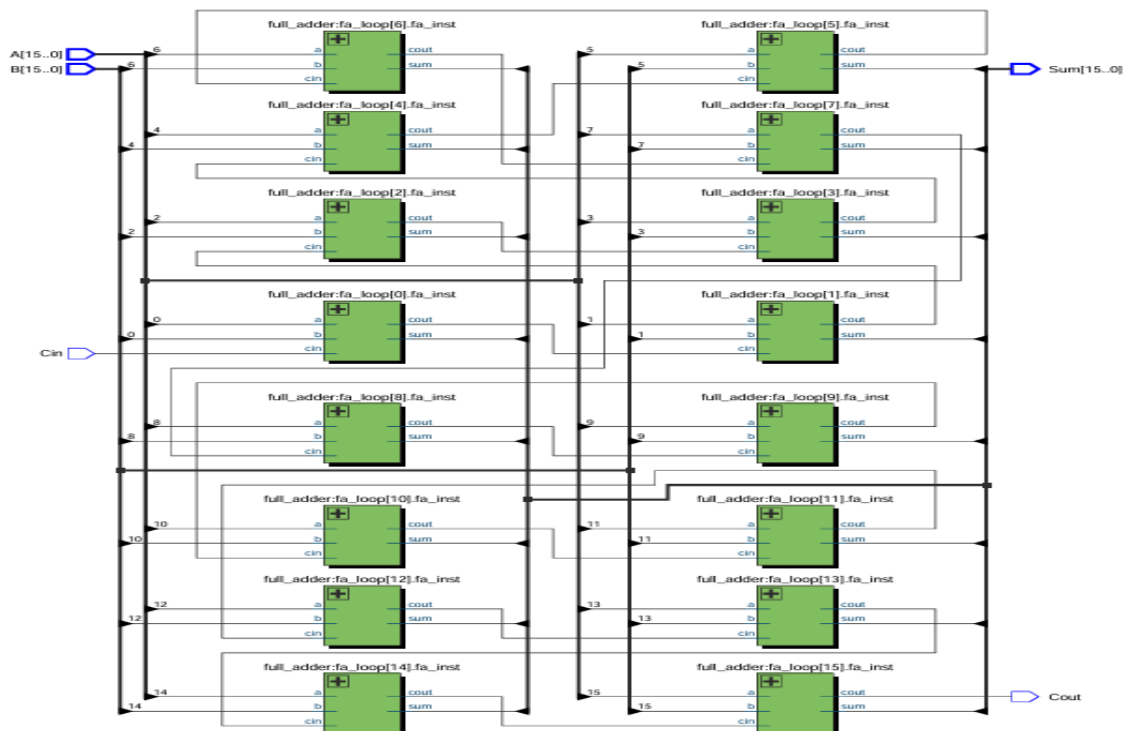
Tất cả kiến trúc bộ cộng trong báo cáo đều được thiết kế và kiểm chứng trên nền tảng chip FPGA Intel Cyclone IV E sử dụng bộ công cụ Intel Quartus Prime. Việc lựa chọn dòng chip này cho phép đánh giá chính xác khả năng tối ưu hóa quá trình biên dịch đối với các cấu trúc logic đặc thù và tài nguyên sẵn có trên phần cứng thực tế.

3.1 Ripple-Carry Adder.

3.1.1 Mô tả kiến trúc.

Kiến trúc RCA được triển khai dựa trên nguyên lý kết nối nối tiếp N khối bộ cộng toàn phần (Full Adder). Trong cấu trúc này, bit nhớ đầu ra (Cout) của tầng trước được nối trực tiếp vào đầu vào nhớ (Cin) của tầng kế tiếp, tạo thành một chuỗi truyền nhớ xuyên suốt từ bit LSB đến MSB. Đây là kiến trúc cơ bản nhất, dễ dàng triển khai nhưng có độ trễ lan truyền lớn do mỗi tầng phải chờ kết quả từ tầng phía trước.

3.1.2 Sơ đồ khối.



Sơ đồ RTL được trích xuất từ phần mềm Quartus Prime cho trường hợp $N = 16$ thể hiện rõ cấu trúc phân cấp:

- Các khối cơ bản: Gồm 16 thực thể full_adder (từ fa_loop[0] đến fa_loop[15]).
- Kết nối dữ liệu: Hai bus đầu vào A[15..0] và B[15..0] được tách ra từng bit tương ứng để đưa vào các khối FA.
- Đường truyền nhớ: Tín hiệu Cin bắt đầu từ khối fa_loop[0] và nối tiếp qua các khối cho đến khi tạo ra Cout cuối cùng ở khối fa_loop[15].

3.1.3 Các tham số thiết kế.

- Độ rộng bit: $N = 16$ (có thể thay đổi linh hoạt qua parameter $N = 8, 32, 64$).
- Chip đích: Intel Cyclone IV E.
- Tối ưu hóa: Thiết kế tận dụng cấu trúc Carry Chain chuyên dụng của FPGA để giảm thiểu độ trễ thực tế trên phần cứng.

3.1.4 Cách tổ chức module.

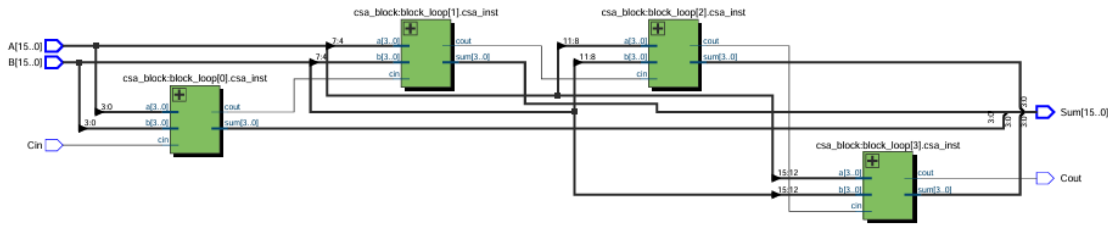
- Module con (full_adder.v): Thực hiện phép cộng 1-bit cho ba tín hiệu a, b, Cin.
- Module tổng thể (ripple_carry_adder.v): Sử dụng cấu trúc vòng lặp generate để khởi tạo đồng loạt 16 khối FA, giúp mã nguồn gọn gàng và dễ dàng mở rộng lên 32 hoặc 64 bit theo yêu cầu khảo sát của bài tập lớn.

3.2 Carry-Skip Adder.

3.2.1 Mô tả kiến trúc.

Kiến trúc Carry-Skip Adder được thiết kế để khắc phục nhược điểm về tốc độ của RCA bằng cách chia N bit thành các nhóm (blocks) có kích thước cố định. Mỗi nhóm được trang bị thêm mạch logic dự đoán truyền nhớ (Skip Logic). Nếu mọi cặp bit trong một nhóm đều thỏa mãn điều kiện truyền nhớ ($P_i = A_i \oplus B_i = 1$), tín hiệu nhớ đầu vào (Cin) của nhóm sẽ được đưa thẳng qua bộ MUX để đến đầu ra của nhóm đó mà không cần chờ đợi quá trình cộng nối tiếp bên trong các khối Full Adder. Điều này giúp rút ngắn đáng kể đường dẫn tới hạn khi số bit N lớn.

3.2.2 Sơ đồ khối.



Lấy ví dụ minh họa $N = 16$, block size = 4.

- Các khối thực thể: Gồm 4 khối csa_block (từ block_loop[0] đến block_loop[3]).
- Phân chia dữ liệu: Các bus đầu vào A[15..0] và B[15..0] được chia thành các cụm 4 bit (như [3:0], [7:4], [11:8], [15:12]) để cấp cho từng khối tương ứng.
- Đường truyền nhớ: Tín hiệu Cin đi vào khối đầu tiên. Đầu ra nhớ của mỗi khối được nối với đầu vào của khối kế tiếp, hình thành chuỗi giúp giảm độ trễ tổng thể.

3.2.3 Các tham số thiết kế.

- Độ rộng tổng quát (N): 16 bit (có thể thay đổi linh hoạt qua parameter $N = 8, 32, 64$).
- Kích thước nhóm (Block size): 4 bit (có thể thay đổi linh hoạt 8 hoặc 16 bit).
- Tham số hóa: Thiết kế sử dụng parameter để dễ dàng cấu hình lại khi khảo sát các trường hợp 8, 32, 64 bit.
- Nền tảng: Tổng hợp trên chip Intel Cyclone IV E.

3.2.4 Cách tổ chức module.

Trong phần mềm Quartus có và chia thành các module như sau:

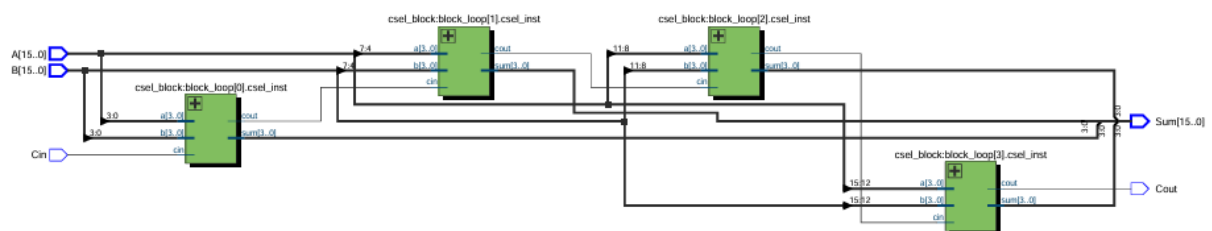
- Module csa_block.v: Chứa một bộ cộng RCA kết hợp với logic MUX và cổng AND để thực hiện chức năng “skip”.
- Module carry_skip_adder.v: Sử dụng vòng lặp generate để kết nối các khối csa_block lại với nhau, đảm bảo tính module hóa và dễ kiểm soát.

3.3 Carry-Select Adder.

3.3.1 Mô tả kiến trúc.

Kiến trúc Carry-Select Adder giải quyết bài toán trễ truyền nhớ bằng cách “đoán trước” cả hai khả năng của bit nhớ đầu vào. Với mỗi nhóm bit (block), mạch sẽ thực hiện đồng thời hai phép cộng song song: một trường hợp giả định $Cin = 0$ và một trường hợp giả định $Cin = 1$. Khi bit nhớ thực tế từ khối trước truyền tới, một bộ MUX sẽ đóng vai trò chọn kết quả đúng trong hai kết quả đã tính sẵn. Cơ chế này giúp loại bỏ thời gian chờ đợi quá trình cộng bên trong block, chỉ còn lại độ trễ của bộ MUX.

3.3.2 Sơ đồ khối.



Lấy ví dụ minh họa $N = 16$, block size = 4.

- Các khối thực thể: Gồm 4 khối `csel_block` (từ `block_loop[0]` đến `block_loop[3]`).
- Cơ chế chọn (Select): Đường `cout` từ khối `block_loop[0]` nối thẳng vào chân `cin` (đóng vai trò tín hiệu chọn) của khối `block_loop[1]`. Quá trình này lặp lại cho các khối tiếp theo.
- Đặc điểm đường dây: Các bus dữ liệu A, B được cấp đồng thời cho các khối, cho phép các phép tính bên trong `csel_block` diễn ra cùng lúc ngay khi có dữ liệu đầu vào.

3.3.3 Các tham số thiết kế.

- Độ rộng bit (N): 16 bit (có thể tham số hóa để khảo sát 8, 32, 64 bit).
- Kích thước nhóm: 4 bit (có thể thay đổi linh hoạt 8 hoặc 16 bit).

- Tài nguyên: Sử dụng nhiều LUT nhất trong 3 kiến trúc do phải chứa gấp đôi số bộ cộng 1-bit bên trong mỗi khối.
- Nền tảng: Tổng hợp trên chip Cyclone IV E.

3.3.4 Cách tổ chức module.

Trong phần mềm Quartus có và chia thành các module như sau:

- Module csel_block.v: Chứa 2 bộ cộng RCA 4-bit và các bộ MUX để chọn Sum[3:0] và Cout.
- Module carry_select_adder.v: Kết nối các khối chọn nhớ theo chuỗi.

3.4 Carry-Lookahead Adder.

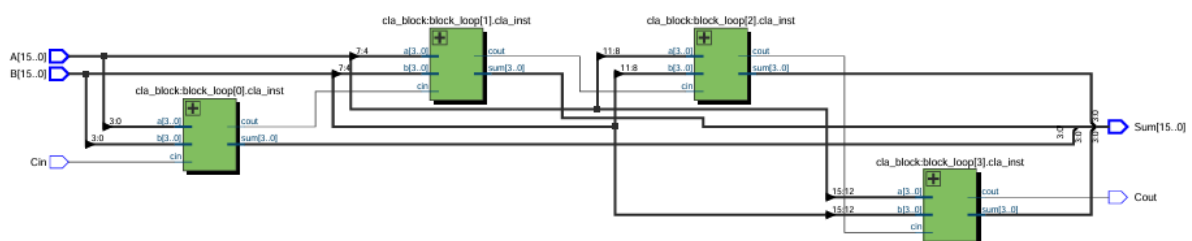
3.4.1 Mô tả kiến trúc.

Bộ cộng CLA khắc phục triệt để bài toán trễ truyền nhớ bằng cách tính toán tất cả các bit nhớ (C_i) song song dựa trên các tín hiệu đầu vào A và B. Kiến trúc này dựa trên hai khái niệm cốt lõi:

Tạo nhớ (Generate – G): $G_i = A_i \cdot B_i$. Bit bit nhớ sẽ được tạo ra nếu cả hai đầu vào đều là 1.

Truyền nhớ (Propagate – P): $P_i = A_i \oplus B_i$ bit bit nhớ sẽ được truyền qua nếu có ít nhất một đầu vào là 1.

3.4.2 Sơ đồ khối.



Lấy ví dụ minh họa $N = 16$.

- Các khối thực thể: Gồm 4 khối cla_block (từ block_loop[0] đến block_loop[3]).

- Tính toán song song: Các bus đầu vào $A[15..0]$ và $B[15..0]$ được phân phối đồng thời đến cả 4 khối. Điều này cho phép quá trình tính toán P và G diễn ra song song ở tất cả các bit.
- Đường truyền nhớ: Khác với RCA, tín hiệu nhớ giữa các khối trong CLA được xử lý bởi logic nhìn trước, giúp giảm thiểu số tầng logic mà bit nhớ phải đi qua, tối ưu hóa đáng kể thời gian xử lý.

3.4.3 Các tham số thiết kế.

- Độ rộng bit (N): 16 bit (thiết kế tham số hóa cho phép mở rộng lên 32, 64 bit).
- Hiệu suất: Mang lại độ trễ thấp nhất trong các kiến trúc khảo sát nhưng cấu trúc logic phức tạp hơn khi số bit tăng cao.
- Nền tảng: Tổng hợp trên chip Cyclone IV E.

3.4.4 Cách tổ chức module.

Trong phần mềm Quartus có và chia thành các module như sau:

- Module `cla_unit.v`: Chứa các phương trình Boolean để tính toán nhanh bit nhớ trung gian.
- Module `cla_block.v`: Kết hợp các bộ cộng bán phần và `cla_unit` để tạo ra kết quả tổng Sum 4-bit.
- Module `carry_lookahead_adder.v`: Kết nối các khối CLA thành một bộ cộng hoàn chỉnh 16-bit.

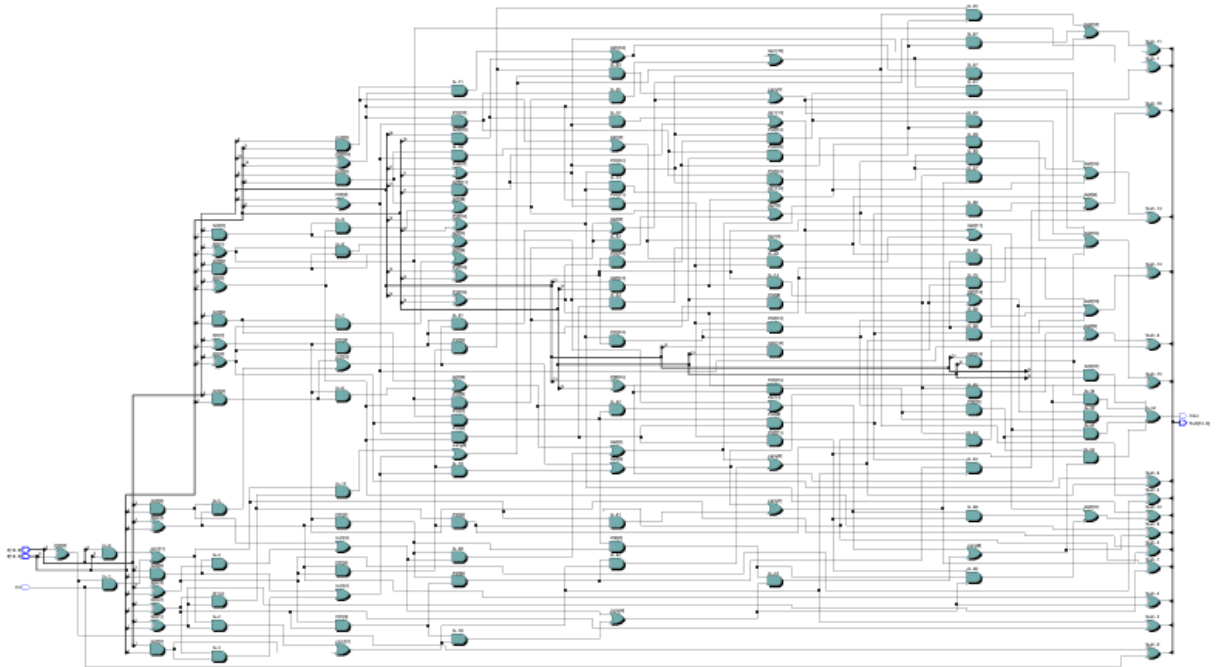
3.5 Parallel Prefix Adder (Kogge–Stone).

3.5.1 Mô tả kiến trúc.

Kiến trúc PPA đại diện cho một bước tiến lớn trong việc tối ưu hóa tốc độ xử lý bit nhớ. Thay vì truyền nhớ tuần tự (như RCA) hay dự đoán theo nhóm phẳng (như CLA), PPA sử dụng cấu trúc mạng lưới cây để tính toán các “tiền tố” bit nhớ. Với cấu trúc này, mọi bit nhớ trung gian đều được tính toán song song thông qua các tầng logic có độ sâu cực thấp. Điều này giúp mạch đạt được tốc độ xử lý nhanh nhất có thể, đặc biệt hiệu quả khi số bit N lớn.

3.5.2 Sơ đồ khối.

Lấy ví dụ $N = 16$



- Mạng lưới tiền tố (Prefix Network): Có nhiều cổng logic và các đường dây tín hiệu dày đặc. Đây chính là các nút tính toán kết hợp để tổng hợp tín hiệu G và P theo từng tầng.
- Số tầng logic: Với $N = 16$, mạch được tổ chức thành 4 tầng xử lý rõ rệt. Mỗi tầng thực hiện việc kết hợp các tiền tố từ tầng trước đó, giúp rút ngắn đường dẫn tới hạn xuống mức tối thiểu.
- Tính đồng bộ: Các đường truyền từ đầu vào đến đầu ra có số lượng cổng logic đi qua gần như tương đương nhau, giúp giảm thiểu hiện tượng lệch pha tín hiệu trên mạch thực tế.

3.5.3 Các tham số thiết kế.

- Độ rộng bit (N): 16 bit (đã tham số hóa để mở rộng khảo sát 8, 32, 64 bit).

- Kiến trúc cụ thể: Triển khai theo cấu trúc cây song song.
- Đặc điểm: Chiếm dụng nhiều tài nguyên do mật độ dây nối và cổng logic cao, nhưng mang lại hiệu năng cao nhất.
- Nền tảng: Tổng hợp trên chip Cyclone IV E.

3.5.4 Cách tổ chức module.

- Module prefix_operator.v: Thực hiện phép toán logic tại mỗi nút cây.
- Module ppa_network.v: Định nghĩa sơ đồ kết nối phức tạp giữa các tầng logic.
- Module parallel_prefix_adder.v: Quản lý toàn bộ quy trình từ tiền xử lý đầu vào đến xuất kết quả tổng cuối cùng.

CHƯƠNG 4: MÔ PHỎNG VÀ KIỂM THỬ

4.1 Mô tả testbench.

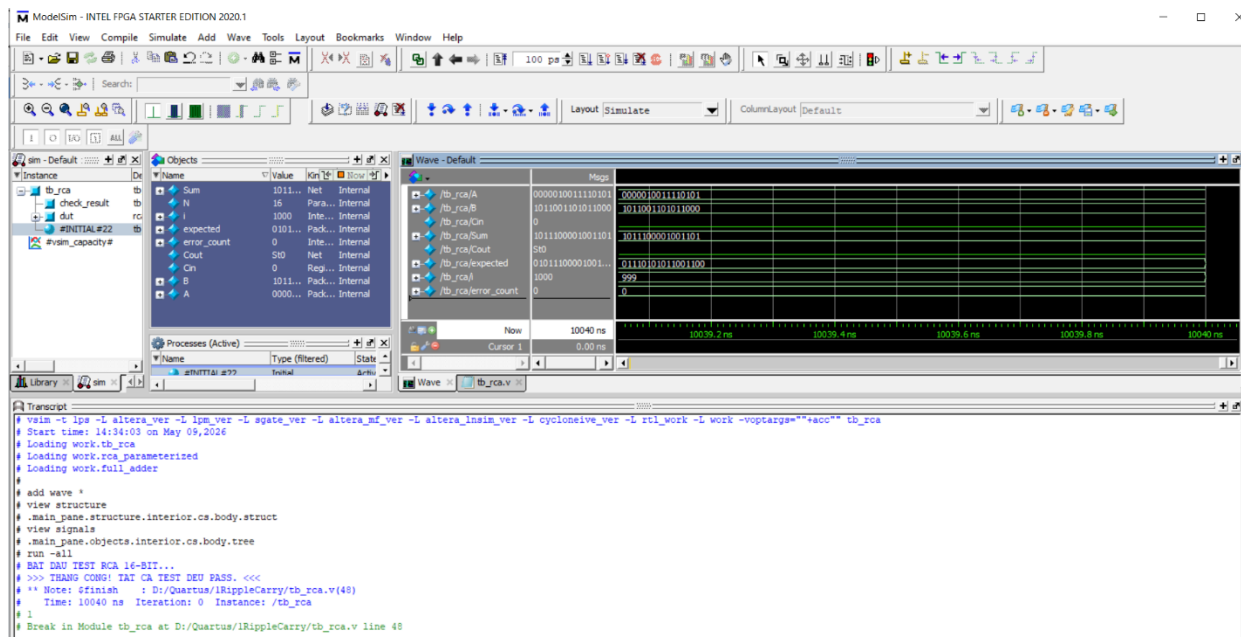
Để kiểm tra tính chính xác của 5 bộ cộng (RCA, CSA-skip, CselA, CLA, PPA), các file testbench của từng bộ cộng đã được xây dựng bằng ngôn ngữ Verilog. Testbench này được thiết kế theo hướng tham số hóa (parameter N) để có thể thay đổi theo các mức bit 8, 16, 32 và 64.

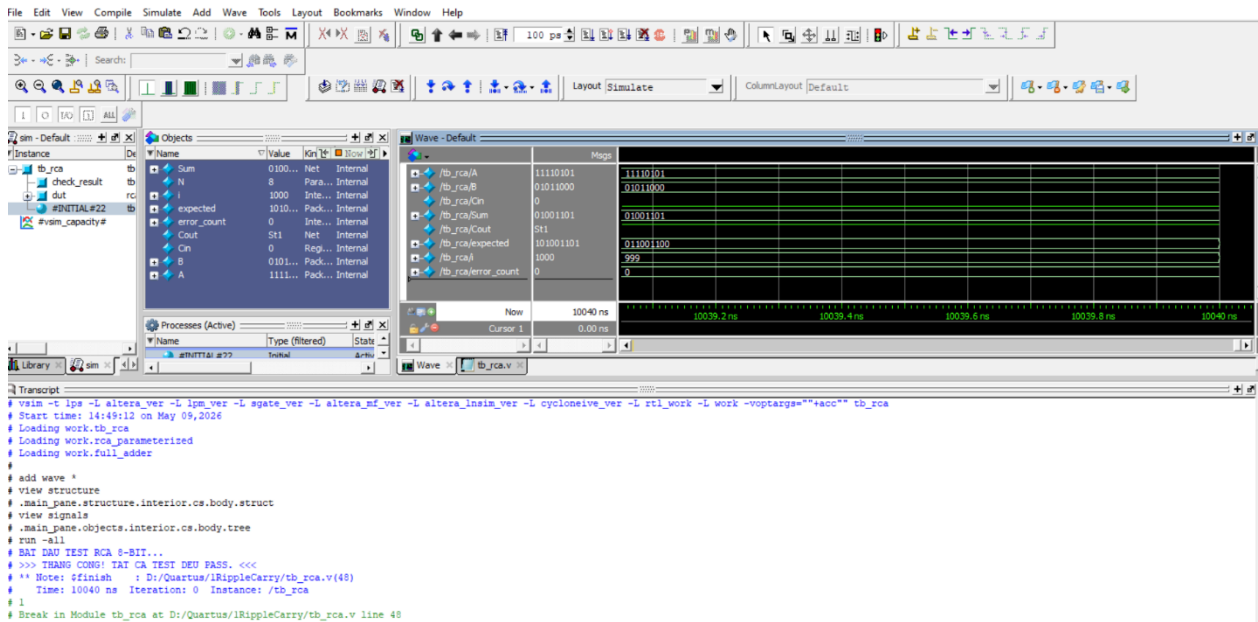
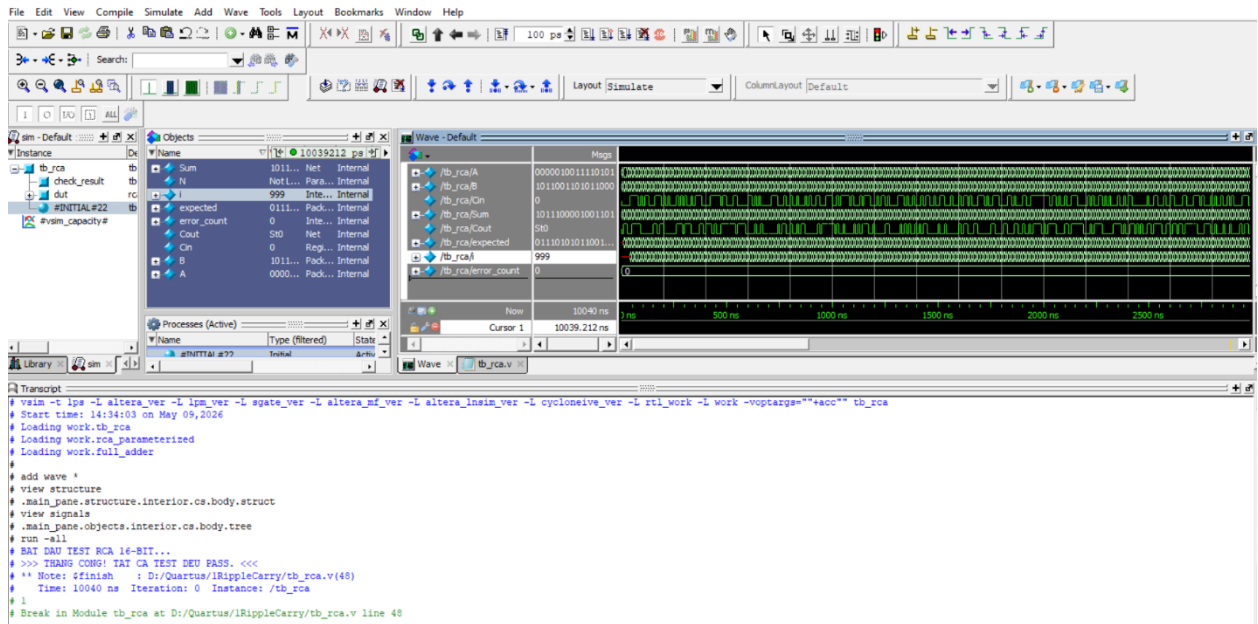
- Cấp dữ liệu đầu vào: Sử dụng vòng lặp hoặc các giá trị biên để cấp dữ liệu cho cả hai bus A và B.
- Kiểm tra tự động: Testbench sử dụng toán tử cộng mặc định của Verilog làm chuẩn đối chiếu. Kết quả của các bộ cộng thiết kế sẽ được so sánh với expected_sum. Nếu sai lệch, Testbench sẽ xuất thông báo lỗi ngay lập tức.
- Công cụ mô phỏng: Sử dụng phần mềm ModelSim – Intel FPGA Edition.

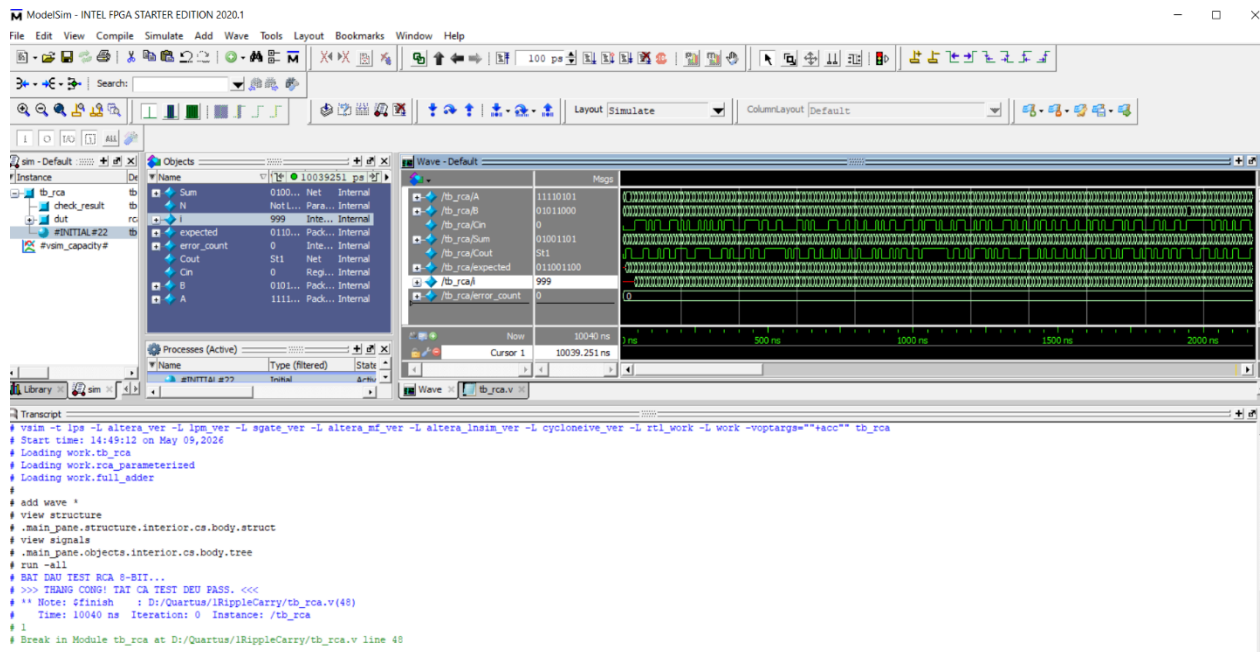
4.2 Các trường hợp kiểm thử và kết quả mô phỏng.

Ripple-Carry Adder

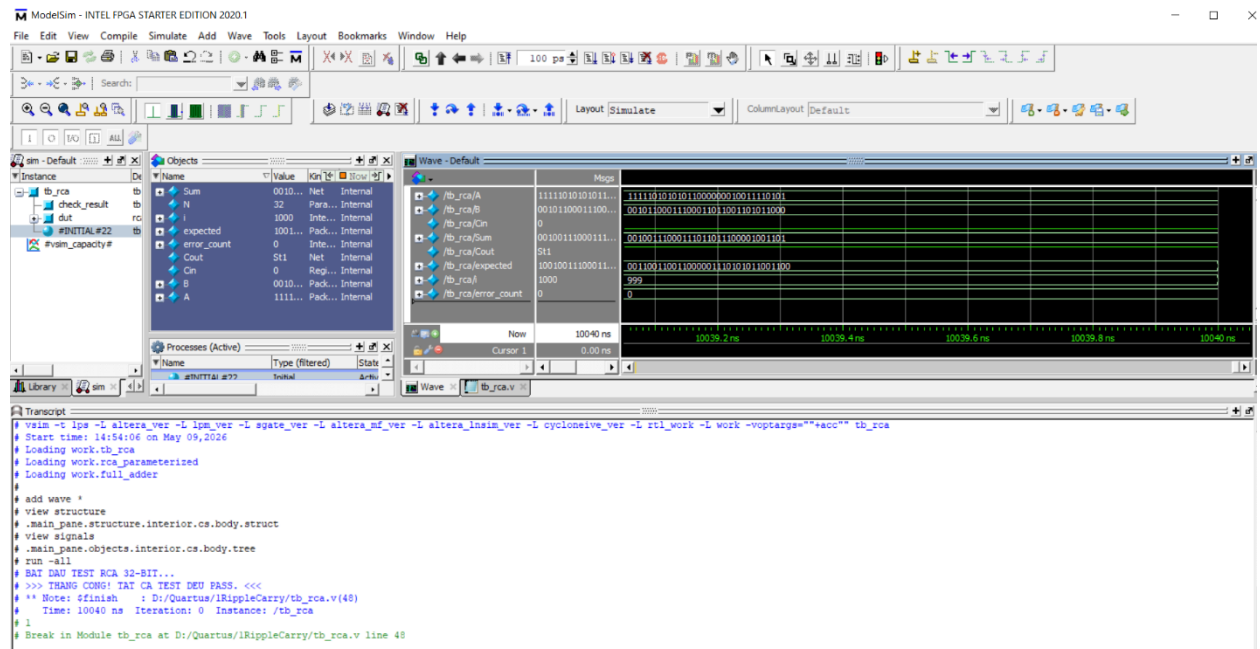
- N = 16

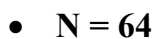


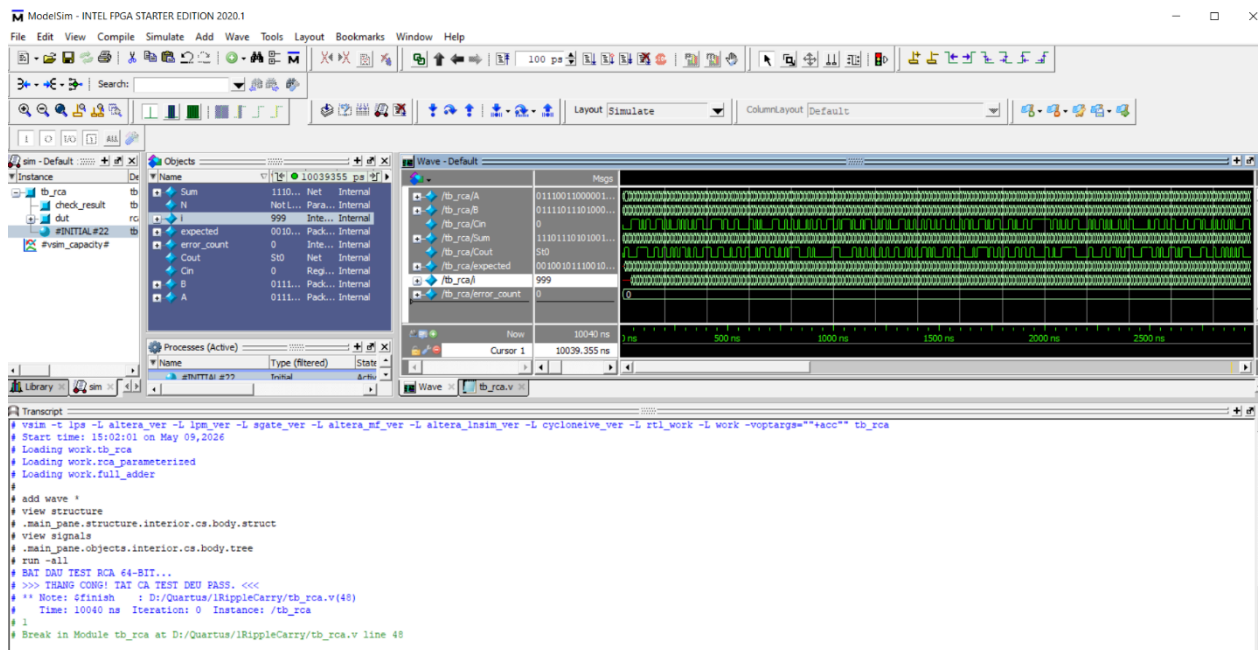




- $N=32$

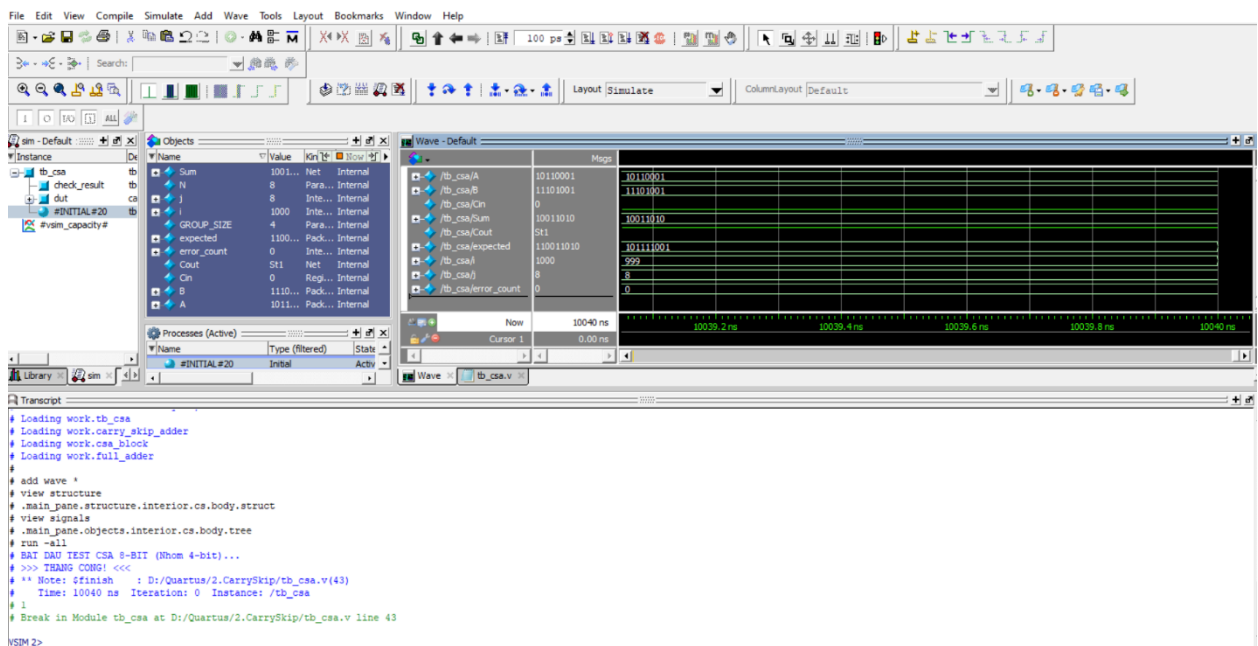


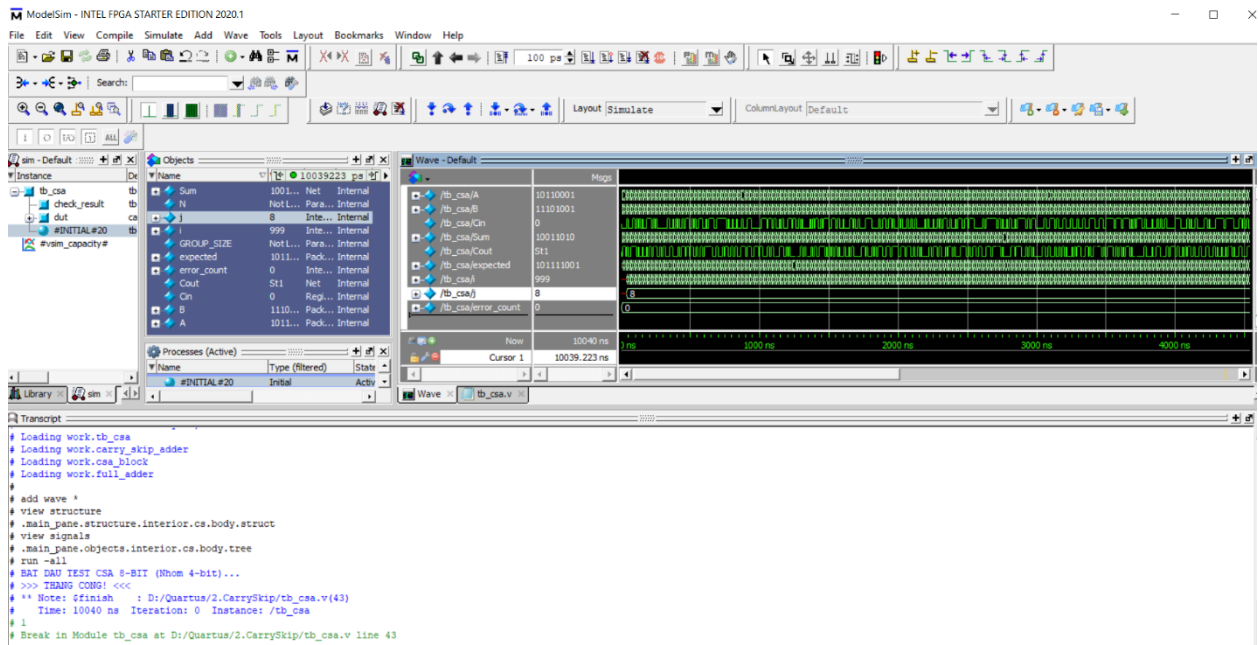




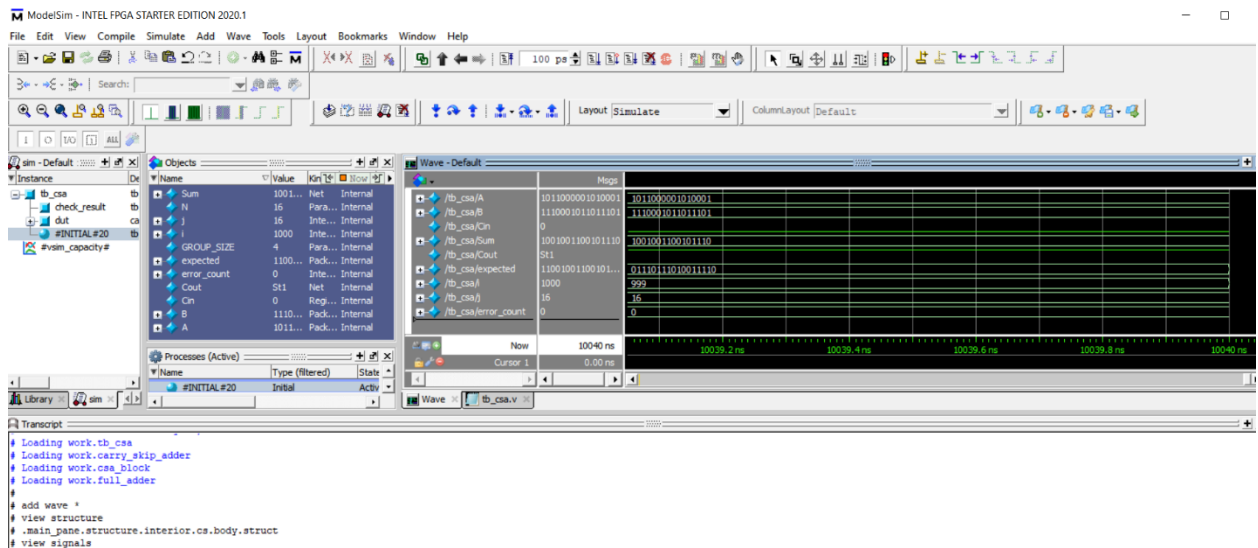
Carry-Skip-Adder

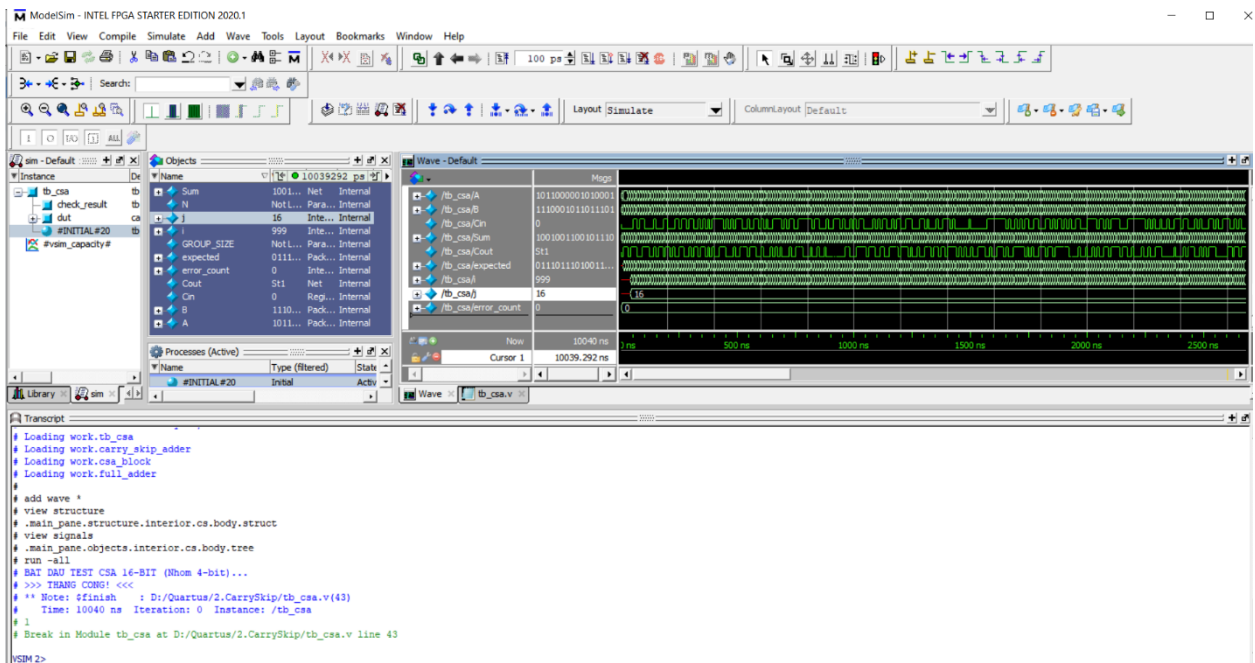
- **N = 8, GROUP = 4**



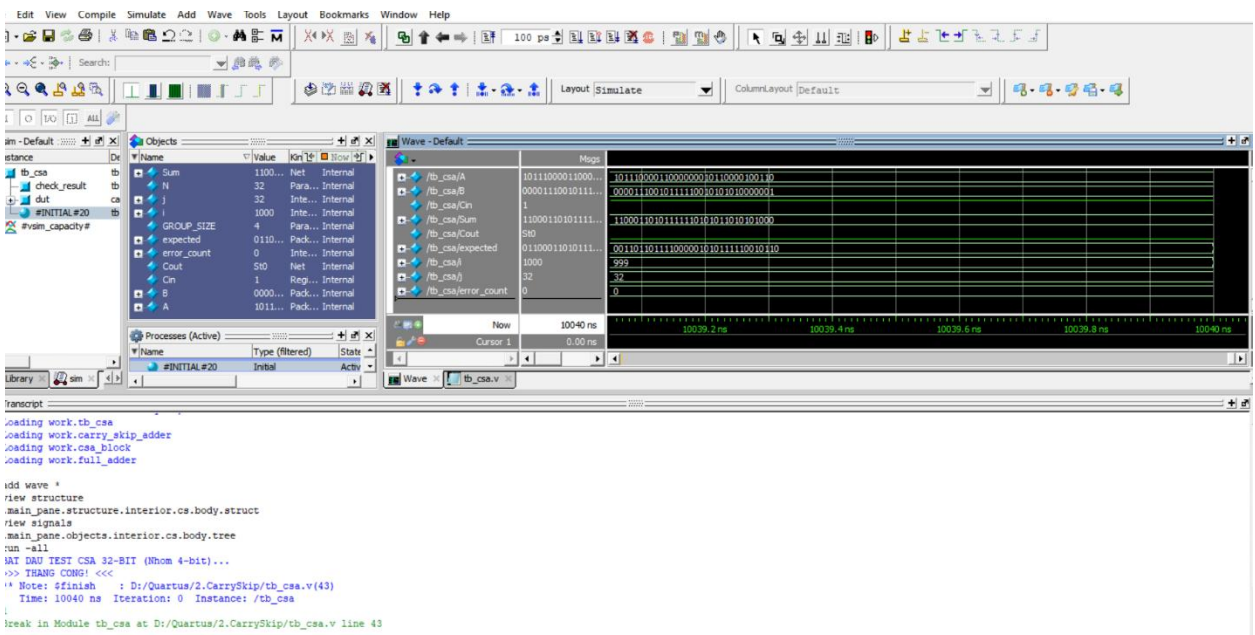


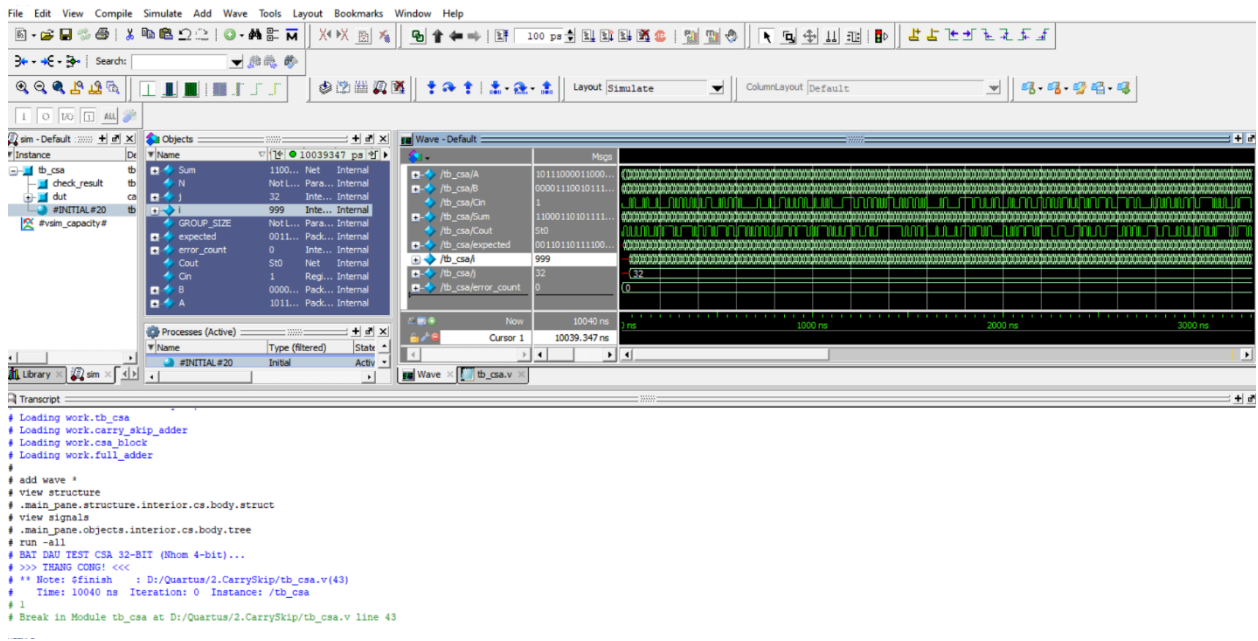
- **N = 16, GROUP = 4**



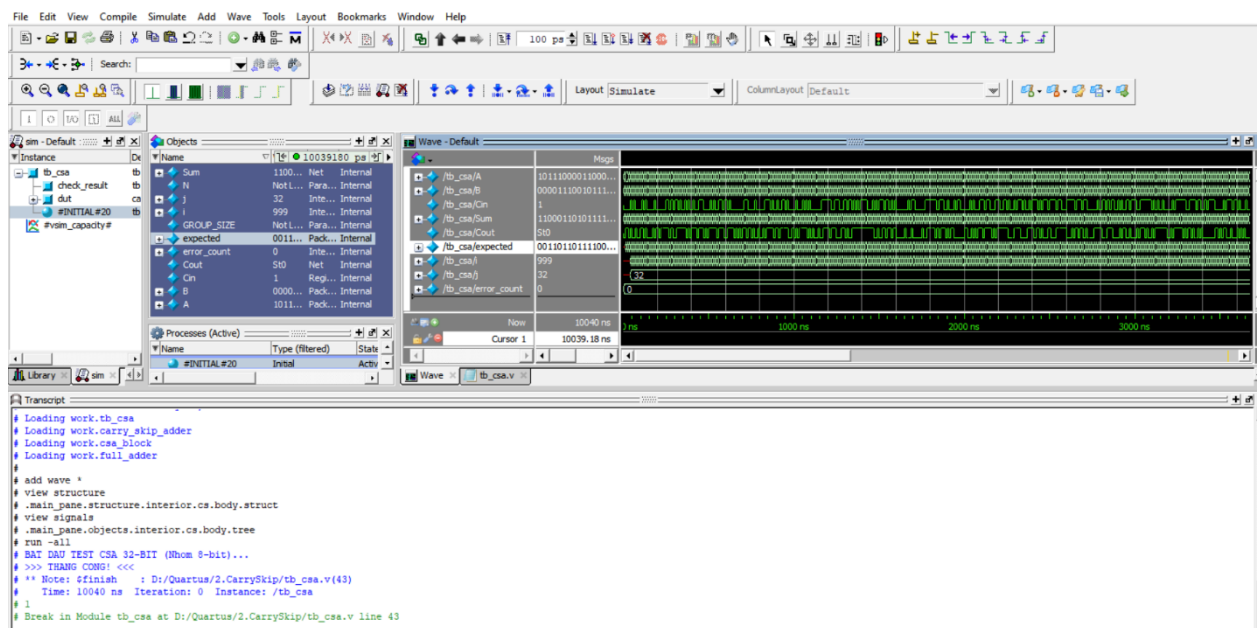


- **N = 32, GROUP = 4**

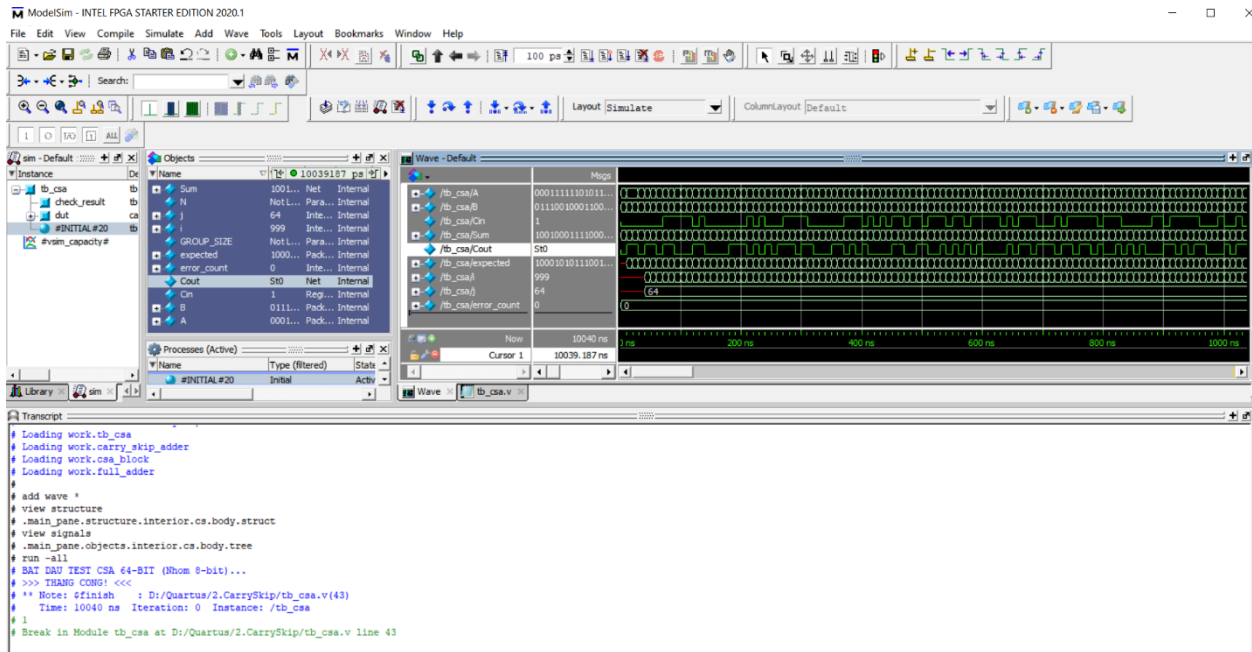




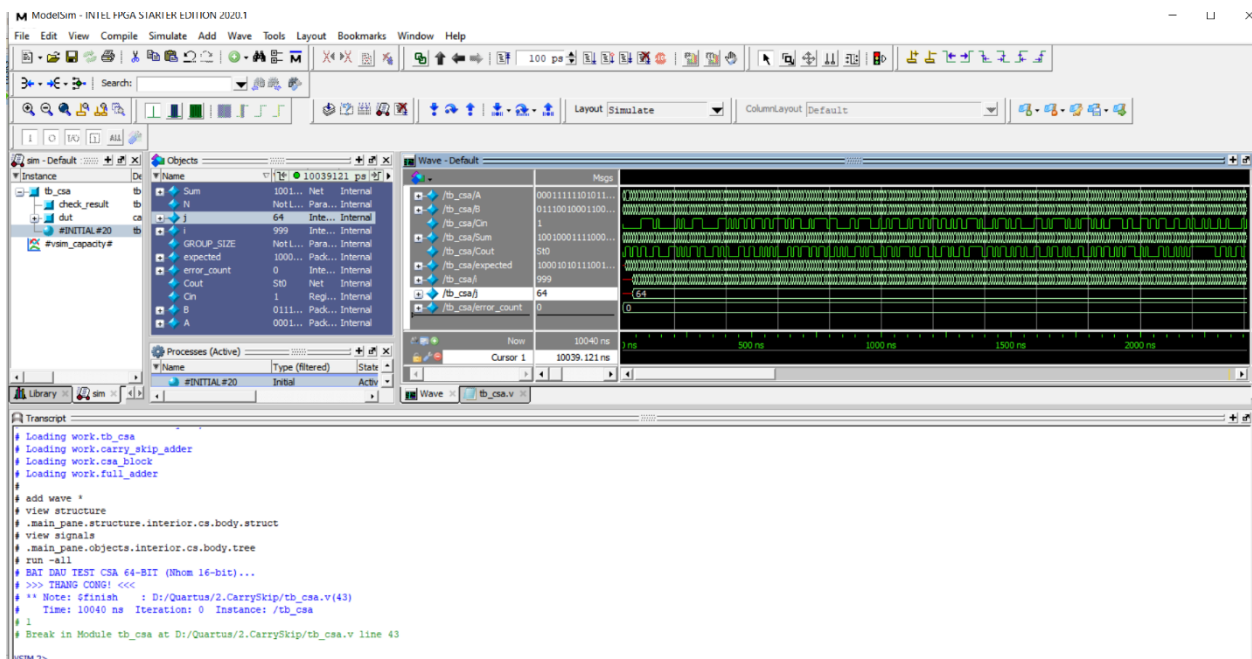
- **N = 32, GROUP = 8**



- **N = 64, GROUP = 8**

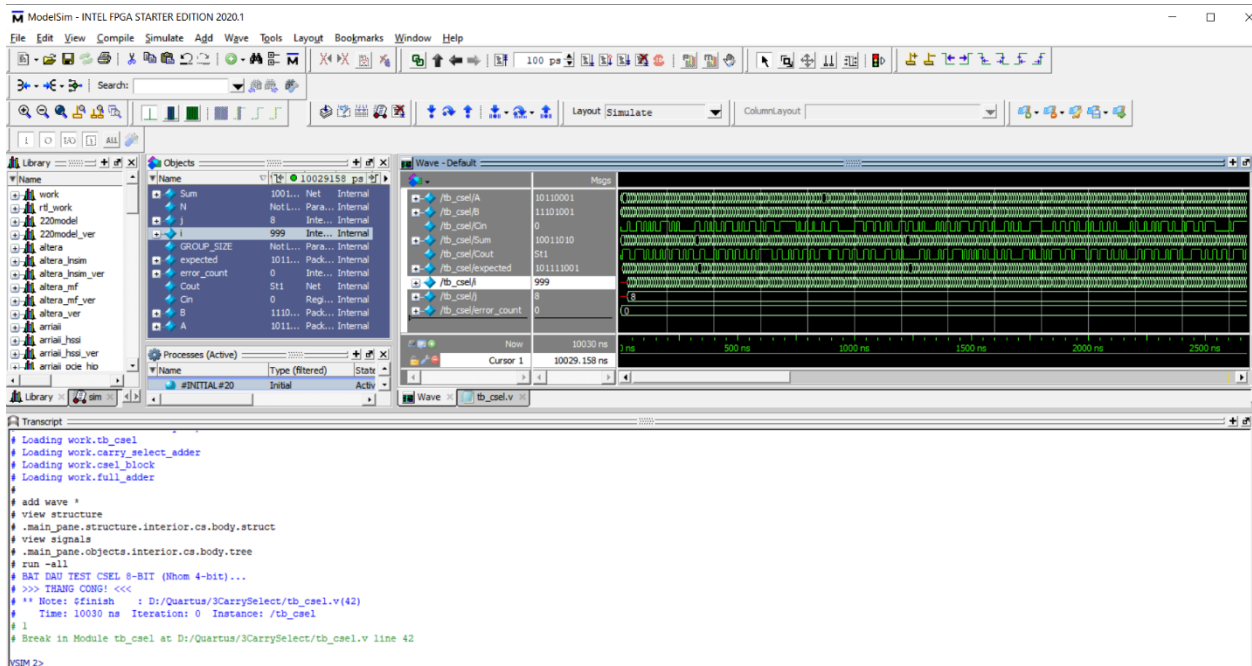


- **N = 64, GROUP = 16**

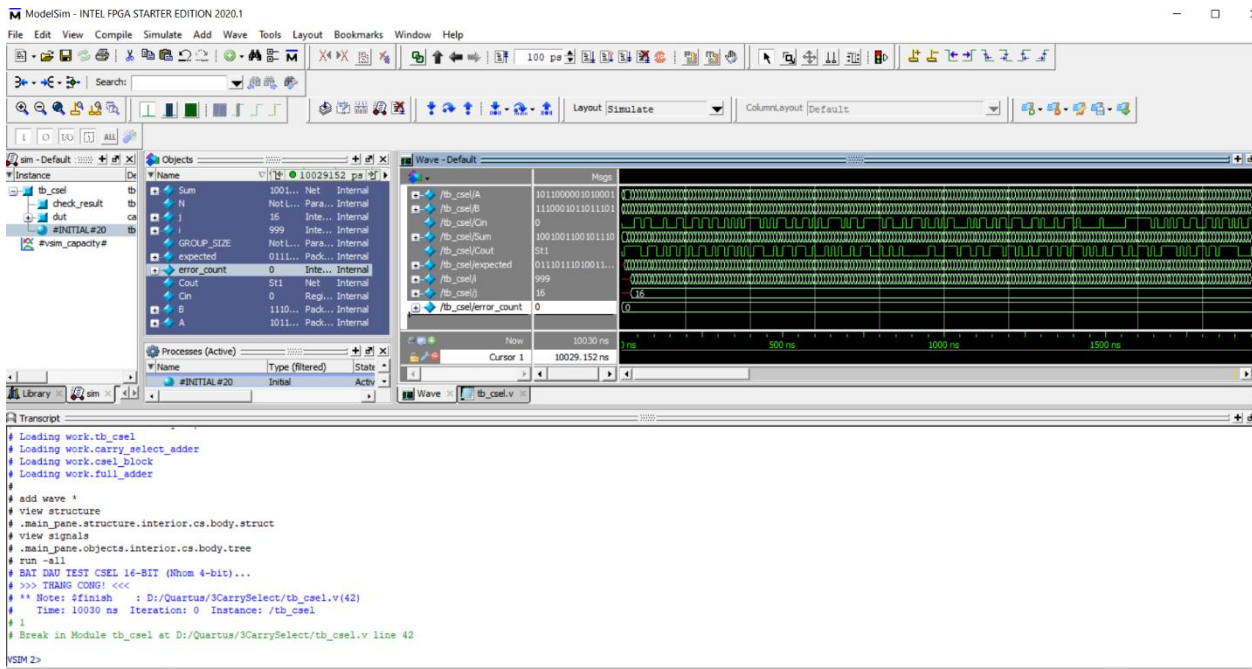


Carry Select Adder.

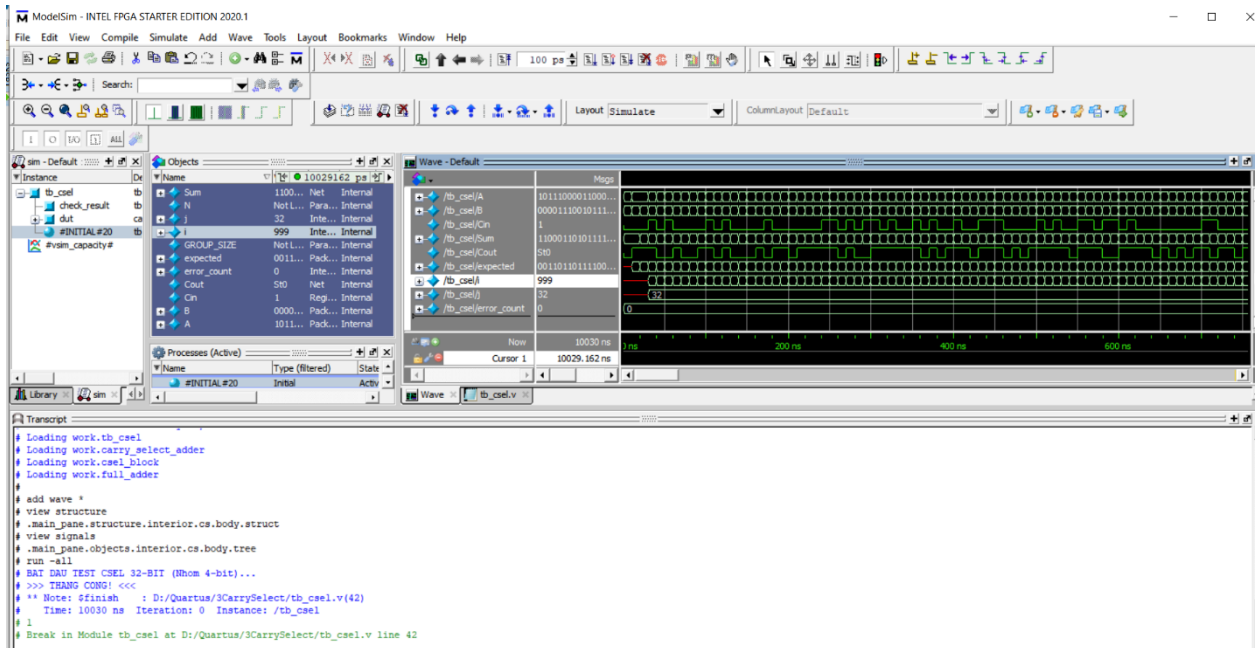
- **N = 8, GROUP = 4**



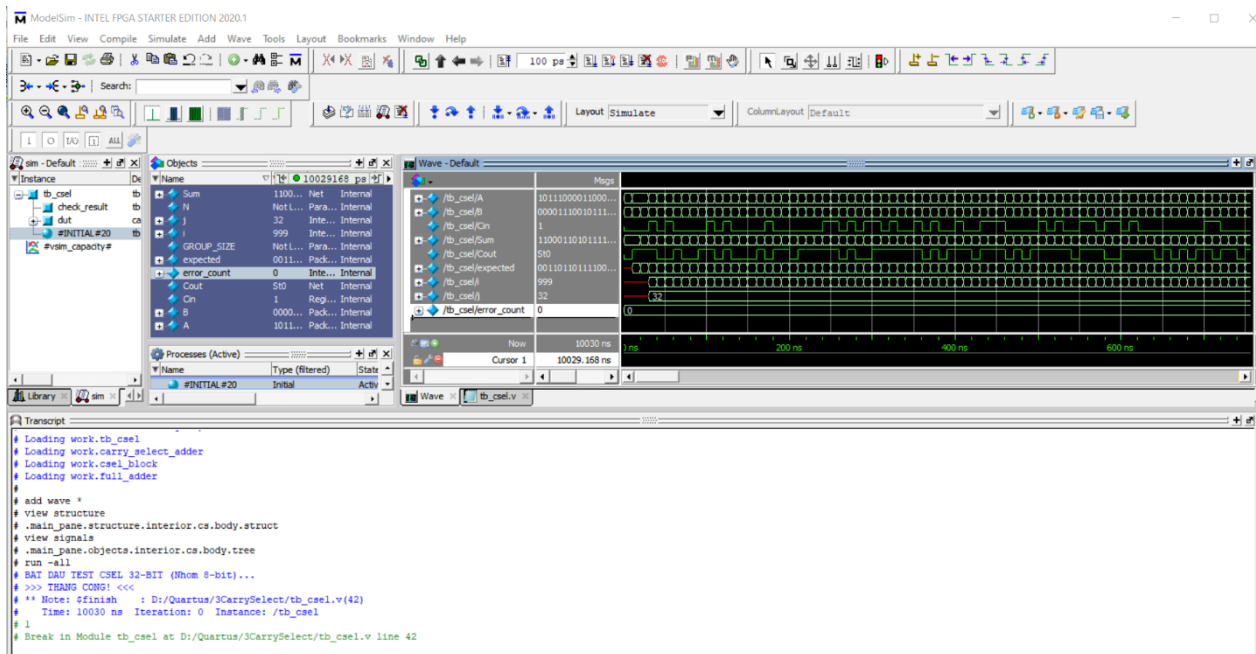
- **N = 16, GROUP = 4**



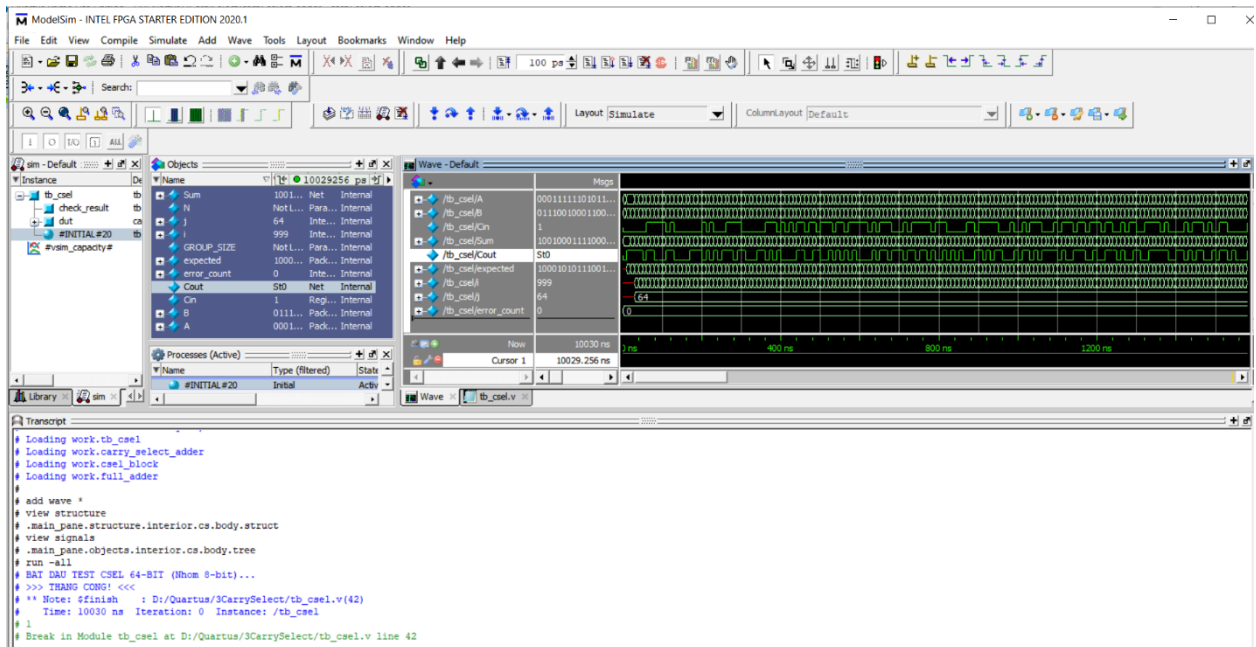
- **N = 32, GROUP = 4**



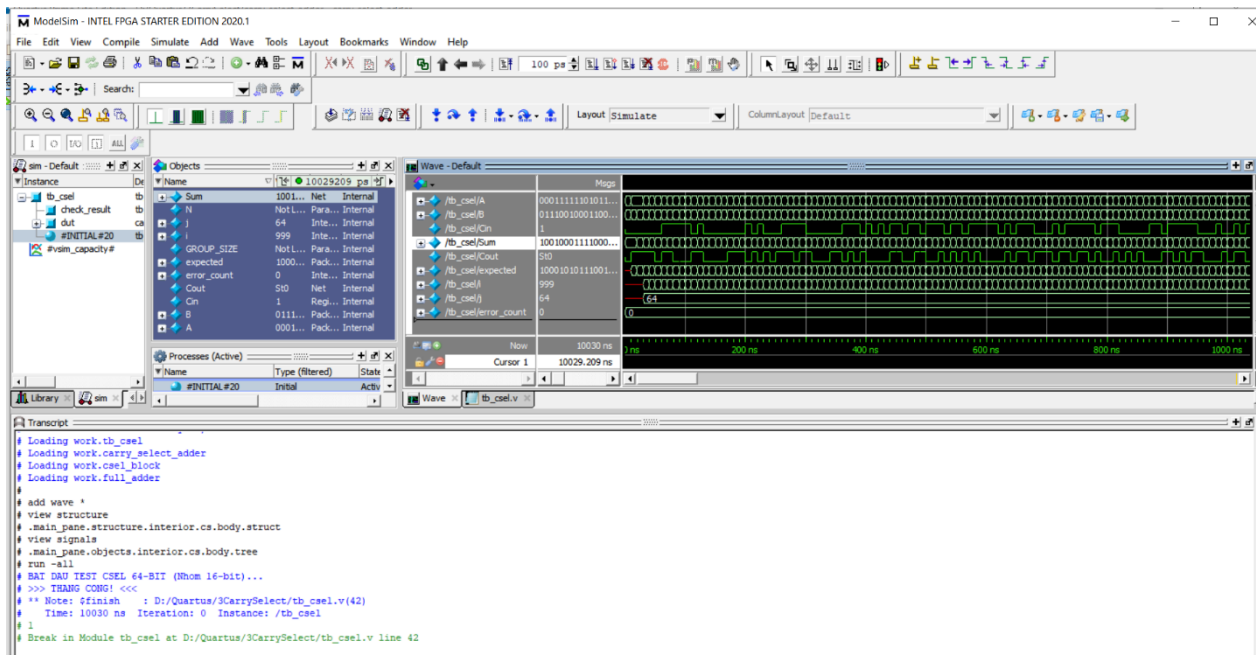
- **N= 32, GROUP = 8**



- **N = 64, GROUP = 8**

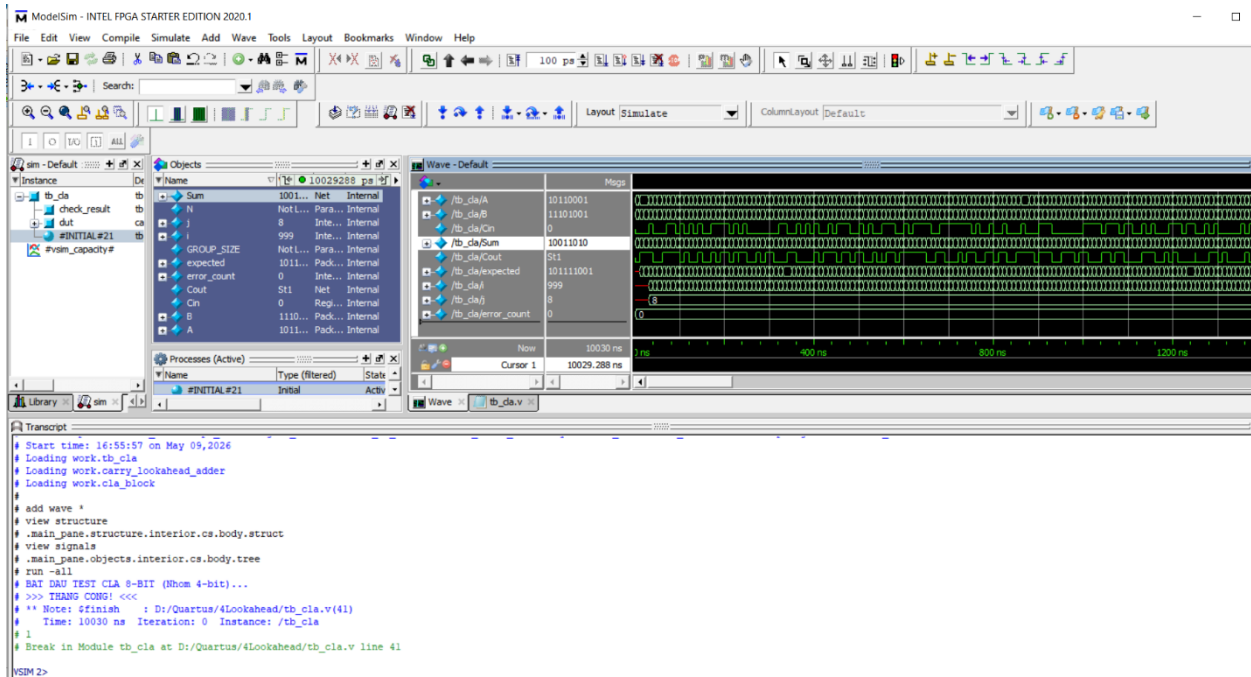


- **N = 64, GROUP = 16**

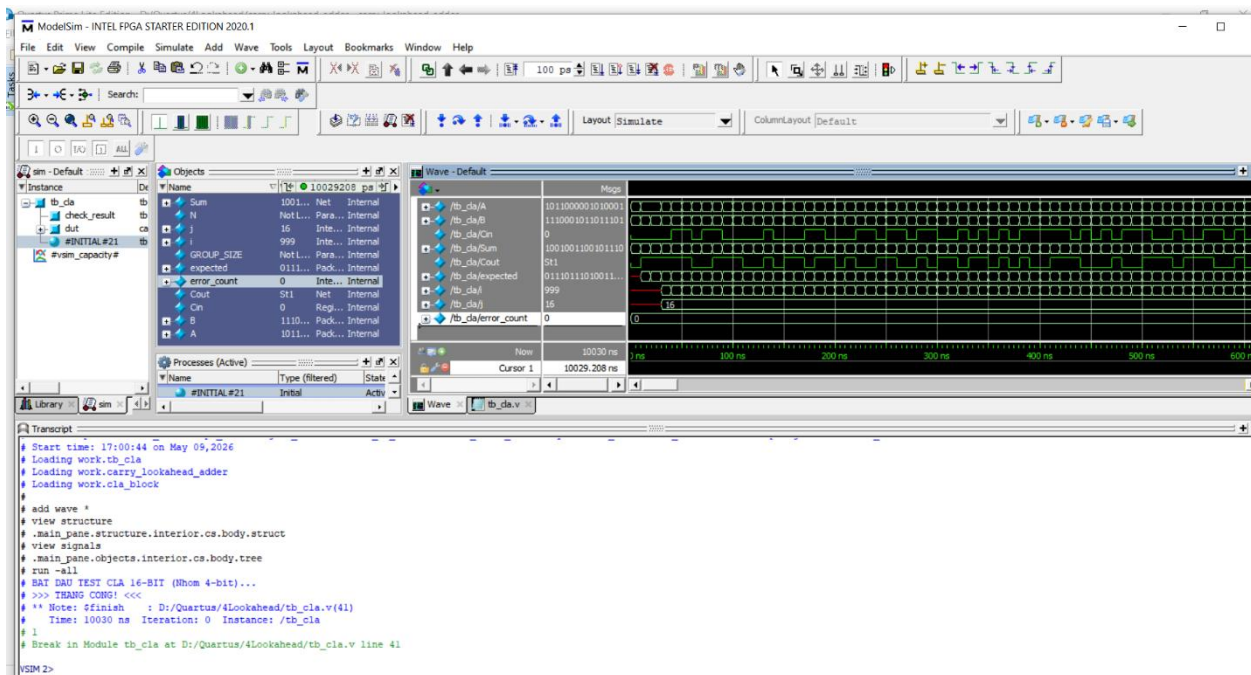


Carry Lookahead Adder

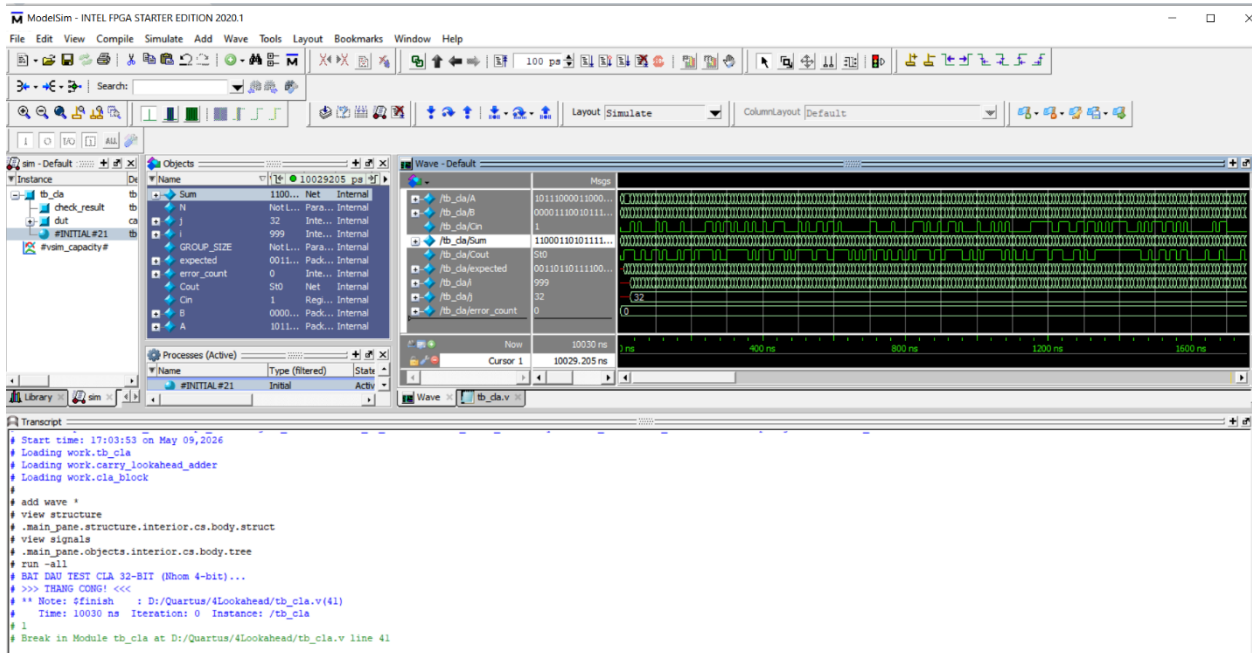
- $N = 8$



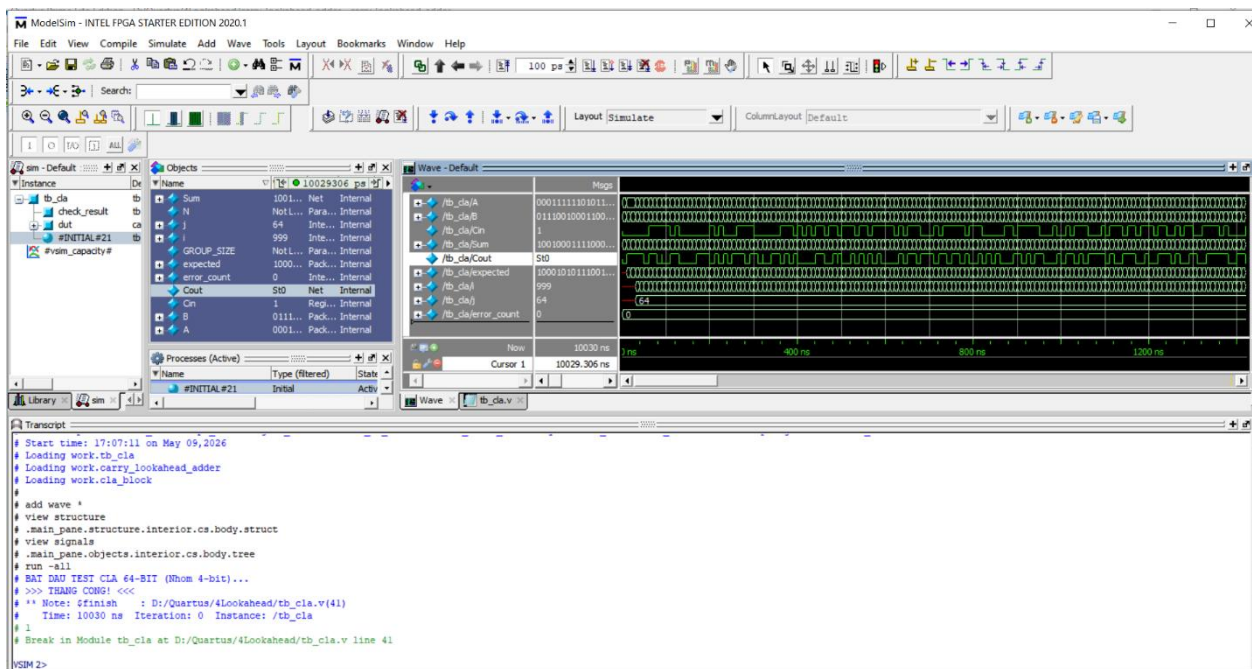
- $N = 16$



- $N = 32$

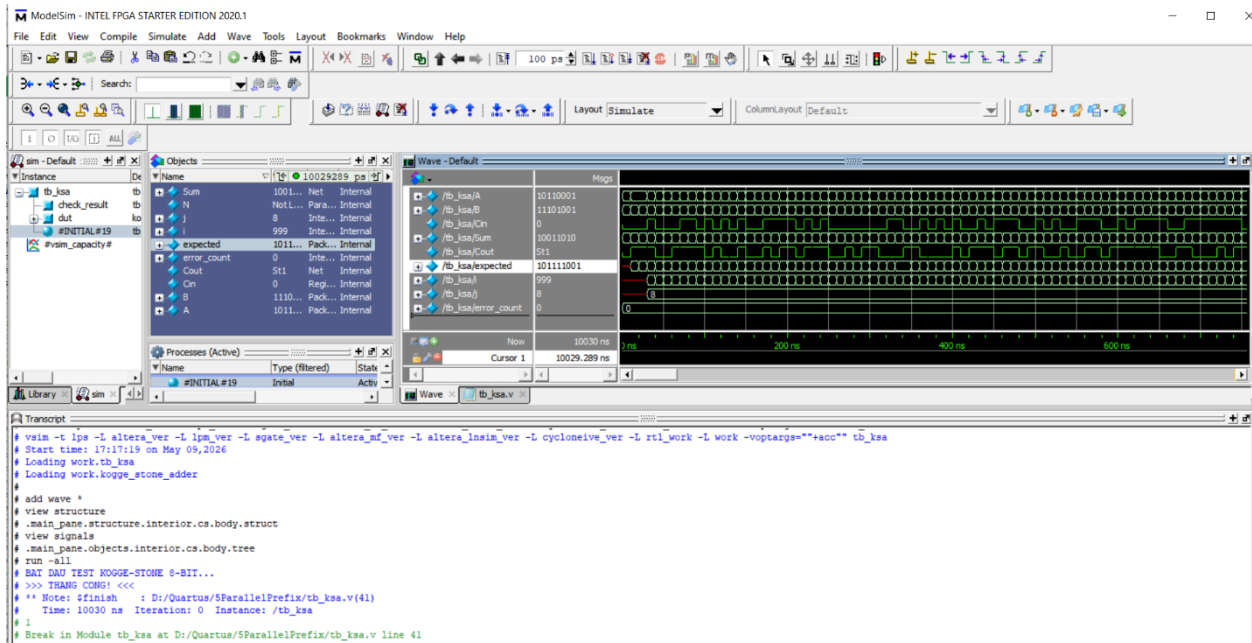


- $N = 64$

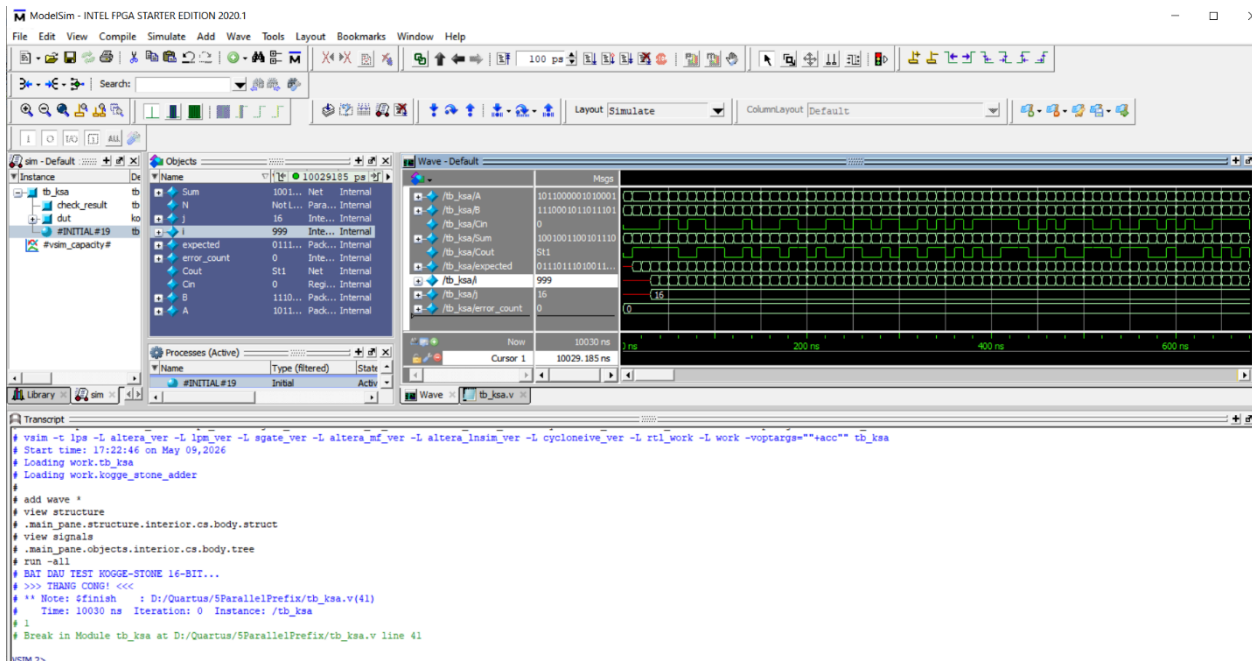


Parallel Prefix(Kogge–Stone)

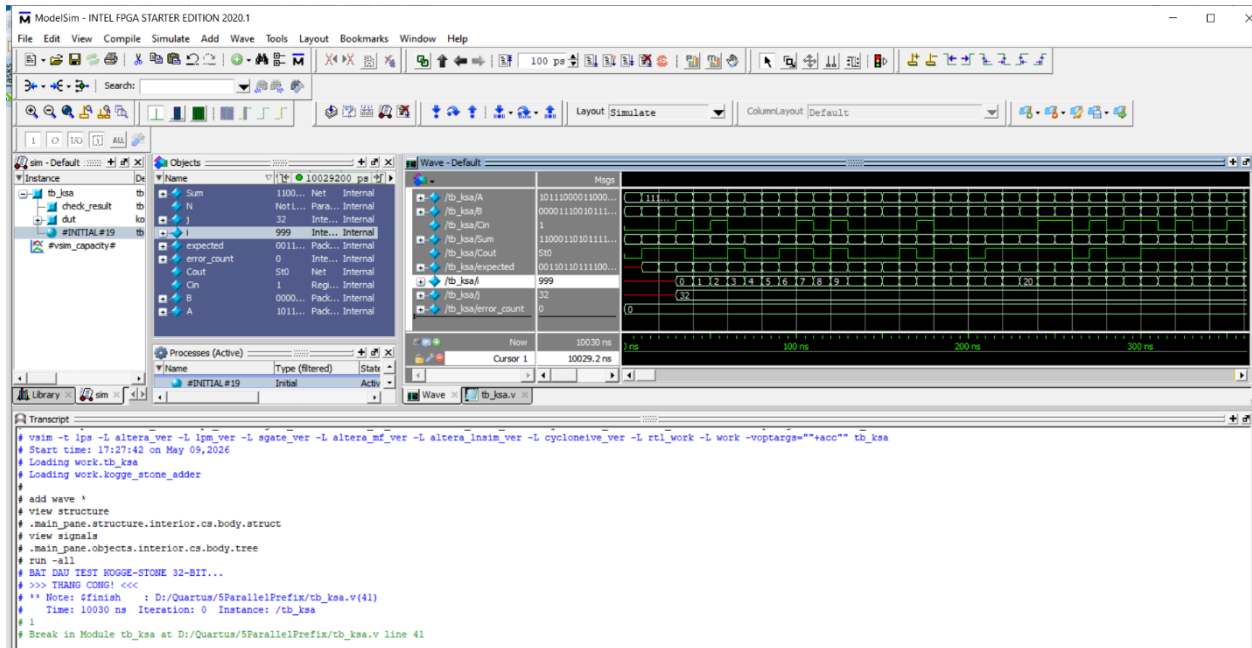
- $N = 8$



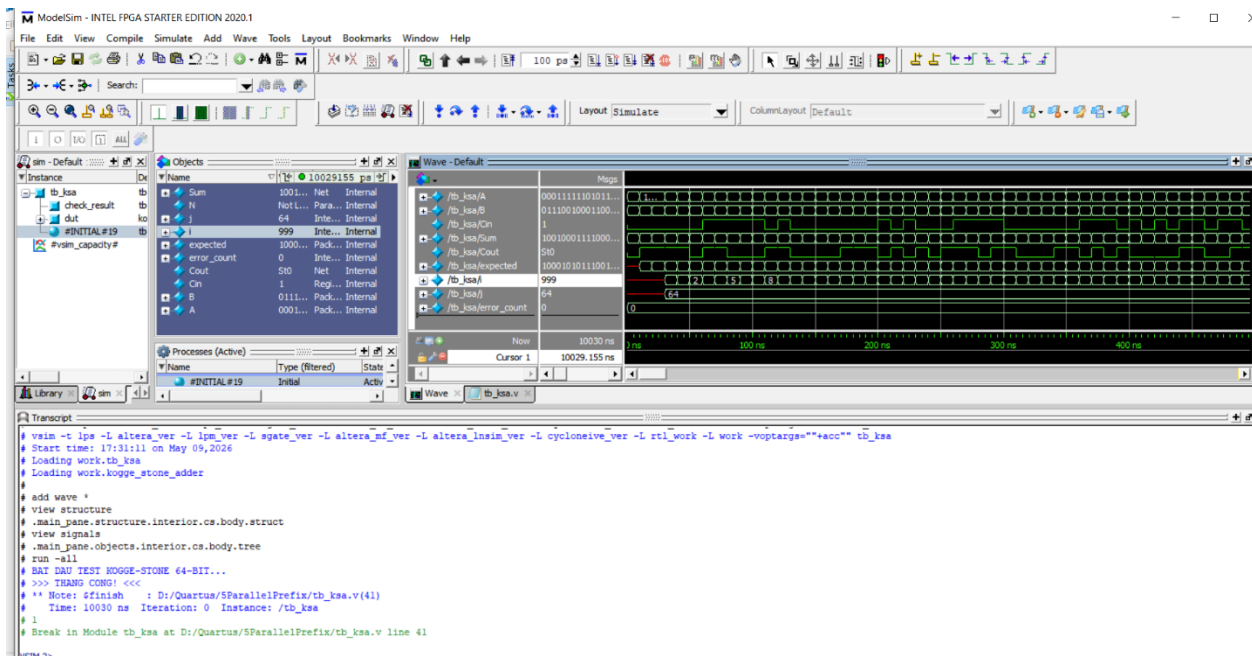
- $N = 16$



- $N = 32$



- $N = 64$



4.3 Kết luận sau khi thu được kết quả mô phỏng.

Sau quá trình triển khai thiết kế và chạy mô phỏng, hệ thống đã đạt được các mục tiêu đề ra trong giai đoạn thực thi:

- **Hoàn thiện mô hình hóa phần cứng:** Toàn bộ 5 kiến trúc bộ cộng từ cơ bản (RCA) đến nâng cao (PPA) đã được mô tả thành công bằng ngôn ngữ Verilog HDL. Việc áp dụng tư duy thiết kế tham số hóa (parameterized design) đã giúp hệ thống trở nên linh hoạt, cho phép cấu hình nhanh chóng các mức độ rộng bit khác nhau (8, 16, 32, 64 bit) mà không cần thay đổi cấu trúc mã nguồn cốt lõi.
- **Xác thực chức năng (Functional Verification):** Thông qua môi trường Testbench chung với các kịch bản kiểm thử ngẫu nhiên và kiểm thử biên, tất cả các module thiết kế đều đã khẳng định được tính đúng đắn về mặt logic. Kết quả thu được từ ModelSim cho thấy sự khớp nối hoàn hảo giữa mạch thiết kế và mô hình toán học chuẩn, không xảy ra sai sót trong quá trình truyền nhớ hay tính toán tổng.
- **Sẵn sàng dữ liệu thực tế:** Quá trình tổng hợp (Synthesis) và tối ưu hóa (Fitter) trên chip FPGA Cyclone IV E bằng phần mềm Quartus Prime đã hoàn tất. Các báo cáo về tài nguyên (Resource Utilization), phân tích thời gian (Timing Analysis) và công suất (Power Analysis) đã được trích xuất đầy đủ và chính xác.

CHƯƠNG 5: TỔNG HỢP VÀ ĐÁNH GIÁ TRÊN FPGA

5.1 Nền tảng FPGA và Công cụ sử dụng.

Nền tảng phần cứng (Target Device):

- Để đảm bảo tính thực tế và chính xác cho các thông số đánh giá, đề tài thực hiện tổng hợp mạch trên chip FPGA của Intel
- Dòng chip: Cyclone IV E.
- Đặc điểm: Đây là dòng chip tối ưu về giá thành và hiệu năng, cung cấp đủ các đơn vị Logic Elements (LEs) để triển khai từ các bộ cộng đơn giản đến các kiến trúc cây phức tạp.

Công cụ phần mềm:

- **Quartus Prime Lite Edition:** Sử dụng để biên dịch, tối ưu hóa sơ đồ RTL và trích xuất dữ liệu phần cứng.
- **TimeQuest Timing Analyzer:** Công cụ chuyên dụng để đo đạc chính xác độ trễ đường dẫn tới hạn (Critical Path Delay) và tần số hoạt động tối đa ($SF_{\max}$).
- **Power Analyzer Tool:** Công cụ ước tính công suất tiêu thụ dựa trên các vector mô phỏng và tài nguyên sử dụng trên chip.

5.2 Kết quả tổng hợp.

RCA (Ripple-Carry Adder)

Kiến trúc	N	LUT	FF	Độ Trễ (nS)	Tần số fmax (1/delay)	Công suất động	Tổng công suất (mW)
RCA	16	32	0	24.595	0.04065867046	0	147.83
RCA	8	16	0	12.976	0.07706535142	0	139.29
RCA	32	72	0	42.806	0.02336121105	0	156.71
RCA	64	154	0	71.198	0.01404533835	0	198.45

Carry-Skip Adder

Kiến trúc	N	Group	LUT	FF	Độ trễ (nS)	Fmax	Công suất động	Tổng công suất(mW)
CSA-SKIP	8	4	22	0	12.875	0.07766990291	0	131.4
CSA-SKIP	16	4	44	0	24.094	0.04150410891	0	144.79
CSA-SKIP	32	4	94	0	53.095	0.01883416518	0	167.64
CSA-SKIP	32	8	96	0	42.975	0.02326934264	0	163.7
CSA-SKIP	64	8	195	0	78.984	0.01266079206	0	182.67
CSA-SKIP	64	16	193	0	70.524	0.01417957008	0	173.25

Carry-Select Adder

Kiến trúc	N	Group	LUT	FF	Độ trễ (nS)	Fmax	Công suất động	Tổng công suất(mW)
Carry-Select Adder	8	4	28	0	12.301	0.08129420372	0	135.35
Carry-Select Adder	16	4	58	0	22.087	0.04527550143	0	142.06
Carry-Select Adder	32	4	108	0	30.516	0.03276969459	0	163.7
Carry-Select Adder	32	8	135	0	27.371	0.03653501882	0	152.46
Carry-Select Adder	64	8	267	0	66.712	0.01498980693	0	177.8
Carry-Select Adder	64	16	304	0	58.514	0.0170899272	0	198.46

Carry-Lookahead Adder.

Kiến trúc	N	LUT	FF	Độ trễ (nS)	Fmax	Công suất động	Tổng công suất(mW)
Carry- Lookahead Adder	8	20	0	13.082	0.07644091118	0	131.41
Carry- Lookahead Adder	16	40	0	22.576	0.04429482636	0	144.81
Carry- Lookahead Adder	32	80	0	47.727	0.02095250068	0	161.27
Carry- Lookahead Adder	64	160	0	75.948	0.01316690367	0	187.84

Parallel Prefix Adder (Kogge–Stone)

Kiến trúc	N	LUT	FF	Độ trễ (nS)	Fmax	Công suất động	Tổng công suất(mW)
Kogge– Stone	8	26	0	11.585	0.08631851532	0	131.39
Kogge– Stone	16	77	0	16.196	0.06174364041	0	148.73
Kogge– Stone	32	221	0	17.486	0.05718860803	0	143.34
Kogge– Stone	64	559	0	24.035	0.04160599126	0	185.4

CHƯƠNG 6: SO SÁNH VÀ THẢO LUẬN

6.1 Về độ trễ (Delay).

- Kiến trúc có độ trễ nhỏ nhất: Dựa trên bảng số liệu, bộ cộng Parallel Prefix Adder và Carry-Lookahead Adder có độ trễ nhỏ nhất. Khi $N = 64$, các kiến trúc này duy trì mức trễ thấp vượt trội so với RCA.
- Sự thay đổi độ trễ khi tăng N : Với RCA, độ trễ tăng theo hàm tuyến tính $O(N)$. Mỗi khi số bit tăng gấp đôi, độ trễ cũng gần như tăng gấp đôi.
 - Với CLA/PPA, độ trễ tăng theo hàm $O(\log N)$, do đó khi N tăng từ 16 lên 64, độ trễ tăng rất chậm.
- Vì sao RCA chậm khi N lớn: Trong RCA, bit nhớ phải truyền tuần tự qua từng Full Adder từ vị trí LSB đến MSB. Đường dẫn tới hạn tỉ lệ thuận với N . Khi $N = 64$, bit nhớ cuối cùng phải đợi qua 64 tầng logic, gây ra trễ tích lũy rất lớn.

6.2 Về tài nguyên phần cứng.

- Kiến trúc sử dụng ít LUT nhất là RCA, đây là kiến trúc tối giản nhất, chỉ cần các cổng logic cơ bản cho mỗi bit mà không cần thêm mạch dự đoán hay chọn nhớ.
- Kiến trúc dùng nhiều tài nguyên nhất: Carry-Select Adder (CselA) hoặc PPA.
- Vì sao Carry-Select tốn diện tích hơn: Kiến trúc này sử dụng cơ chế tính toán song song cho cả hai trường hợp $C_{in} = 0$ và $C_{in} = 1$. Thực chất, nó sử dụng gần như gấp đôi số lượng bộ cộng RCA 1-bit bên trong mỗi khối, cộng thêm các bộ MUX để chọn kết quả, dẫn đến số lượng LUT tăng vọt.

6.3 Về tần số hoạt động cực đại (F_{max}).

- Kiến trúc đạt F_{max} cao nhất: là PPA hoặc CLA.
- Mối quan hệ giữa Critical Path và F_{max} : Tần số hoạt động cực đại tỉ lệ nghịch với độ trễ của đường dẫn tới hạn ($F_{max} = 1/T_{delay}$). Một kiến trúc có đường dẫn tới hạn ngắn (ít tầng logic trung gian) sẽ cho phép hệ thống vận hành ở tần số cao hơn. Do PPA/CLA rút ngắn đường truyền nhớ nên F_{max} của chúng luôn đứng đầu.

6.4 Về công suất (Power).

- Sự thay đổi công suất động: Công suất động tỉ lệ thuận với số lượng cổng logic chuyển mạch và tần số hoạt động. Các kiến trúc phức tạp như CSelA hay PPA có nhiều cổng logic hoạt động đồng thời hơn nên công suất động thường cao hơn RCA.
- Kiến trúc nhanh nhất có tiết kiệm công suất không: Không nhất thiết, thường thì các kiến trúc nhanh nhất (như PPA) lại tiêu tốn nhiều năng lượng hơn do mật độ cổng logic lớn và tần số cao.
- Đánh đổi (Trade-off): Có một sự đánh đổi rõ rệt: Nếu muốn Delay nhỏ (nhanh) thì phải chấp nhận Area lớn (tốn LUT) và Power cao. RCA nằm ở cực tiết kiệm diện tích nhưng chậm, trong khi PPA nằm ở cực tốc độ nhưng tốn diện tích.

6.5 Về tối ưu hóa đặc thù trên FPGA (FPGA-specific optimization).

- Carry chain chuyên dụng: Các dòng FPGA của Intel (Altera) như Cyclone IV có cấu trúc carry chain chuyên dụng (dedicated carry circuitry).
- Ảnh hưởng đến RCA: Nhờ có carry chain, các bộ cộng RCA trên FPGA thực tế chạy nhanh hơn nhiều so với lý thuyết cổng logic rời rạc. Tuy nhiên, khi N quá lớn (như 64 bit), carry chain vẫn bị giới hạn bởi trễ vật lý của đường dây dài trên chip.
- Kết quả lý thuyết và tổng hợp: Kết quả tổng hợp thường tốt hơn lý thuyết (đặc biệt với RCA) vì trình biên dịch Quartus thực hiện tối ưu hóa mức logic (logic mapping) và tận dụng các khối phần cứng cứng (hard IP) có sẵn trong FPGA thay vì chỉ sử dụng các cổng logic rời rạc. Tuy nhiên, xu hướng so sánh giữa các kiến trúc (PPA nhanh hơn RCA) vẫn giữ nguyên đúng như lý thuyết.